Haaga-Helia

CSC Rahti 2 Handbook

Deploying applications to CSC's container cloud — browser, CLI and agent

A community guide, not official CSC documentation. Latest version and agent skills:
https://github.com/laguagu/csc-rahti-guide
Generated 2026-08-31

1. Getting started: access, login, and projects

> What Rahti is, how to get access, how to log in from the browser and the command line, and what resources your project can use.

Contents

What Rahti 2 is

Rahti 2 is CSC's container cloud. It runs on **OKD**, the open-source community edition of Red Hat OpenShift. In practice: you write a Dockerfile, push the image to a registry, and Rahti runs it as a pod, gives it a public HTTPS address, and restarts it if it crashes.

Rahti is a good fit for	Rahti is not a good fit for
Web apps and APIs	GPU computing and model training → Roihu or LUMI
Always-on services	Batch jobs and Slurm jobs → Roihu
Microservices and background services	Managing virtual machines → cPouta
Demos and teaching	Containers that need root privileges (not possible, see below)

What if Rahti isn't the right fit?

Rahti is free for a CSC project and keeps data in Finland, which is hard to beat for teaching use. It isn't always the fastest route to everything, though:

Platform	When it beats Rahti	Watch out for
Vercel	A Next.js/React frontend, preview URLs on every PR, edge functions	The free tier doesn't allow commercial use; data is in the US by default
Render	A backend service + managed Postgres without Kubernetes	Free-tier services sleep when idle
Railway	The fastest "repo in, URL out" for experiments	Usage-based pricing can catch you out
Hetzner or cPouta	You need your own VM, a GPU, or root privileges	Maintenance, updates, and security are on you

In practice, the same Dockerfile works on all of these — the difference is who handles the configuration. On Rahti you write the Kubernetes objects yourself; on Vercel and Railway the platform guesses them for you. For teaching purposes, Rahti teaches more, because the objects stay visible.

A common combination in student projects is a frontend on Vercel with a heavier backend + database on Rahti: you get a public demo URL easily, while the data and models stay on CSC's side.

Key addresses:

What	Address
Web console	`https://console.rahti.csc.fi`
API server	`https://api.2.rahti.csc.fi:6443`
Internal container registry	`image-registry.apps.2.rahti.csc.fi`
App URL space	`*.2.rahtiapp.fi`
Egress IP (for firewall rules)	`86.50.229.150` — can change, always check the documentation

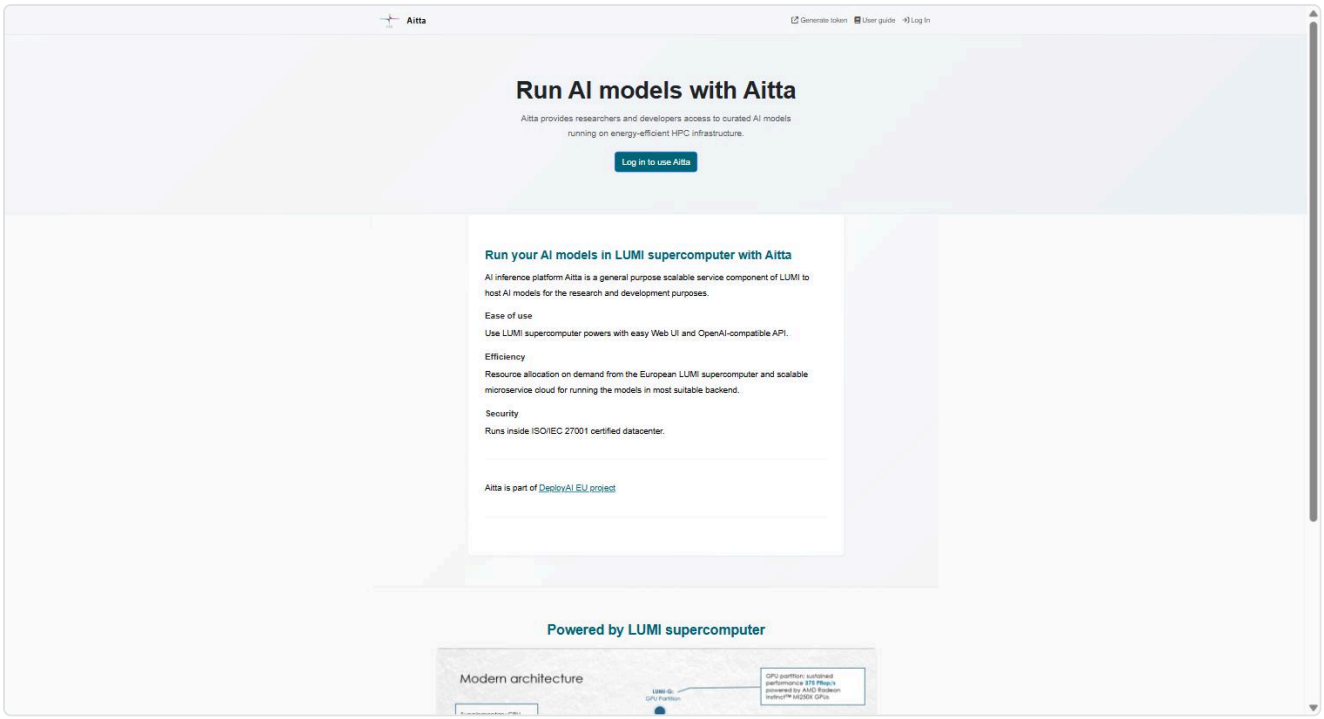
CSC's other services on the same project

Rahti access opens the door to CSC's other services on the same computing project. These often turn up in the same application:

Service	For	Guide
Satama	Container registry (Harbor), sharing images across projects, vulnerability scanning	chapter 2
Allas	Object storage over S3, large files and backups	chapter 8
Aitta	Ready-made language and embedding models behind an OpenAI-compatible API	csc-rahti skill
Roihu	NVIDIA GH200 GPUs, model training and batch jobs	csc-roihu skill
LUMI	AMD MI250X, EuroHPC allocation	csc-lumi skill

Aitta is particularly handy for teaching: you get language and embedding models behind an API without your own GPU and without a commercial API key. The models run on the LUMI

supercomputer and the API is OpenAI-compatible, so existing code works by changing `baseURL`.




```
from openai import OpenAI

client = OpenAI(
    base_url="https://aitta-api.csc.fi/openai",    # AITTA_API_BASE
    api_key="<AITTA_API_TOKEN>",                    # token: https://aitta-auth.csc.fi
)
```

The catalogue includes Llama 3.3, gpt-oss, Gemma 3, Qwen3-VL, embedding models and the Finnish **Poro 2**. Most models are "offline", meaning they have to be woken with `POST /model/{id}/preload` before the first chat completion.

Prerequisites

1. **A CSC user account** — created at MyCSC.
2. **A computing project with Rahti enabled:**
   - Log in at my.csc.fi → *My Projects*
   - Select your project → open **Rahti** in the service list → accept the terms of use → *Apply for access*
   - CSC confirms the application. Permission sync can take about 10 minutes.
3. **Docker or Podman** locally, if you build images yourself.
4. **The `oc` command-line tool**, if you don't want to do everything in the browser.

Installing the tools, briefly:

Tool	Windows	macOS	Linux
Docker	Docker Desktop or `winget install Docker.DockerDesktop`	Docker Desktop or `brew install --cask docker`	`sudo apt install docker.io` + `sudo usermod -aG docker $USER`
Podman (lighter alternative)	`winget install RedHat.Podman`	`brew install podman`	`sudo apt install podman`
`oc`	`scoop install openshift-cli`	`brew install openshift-cli`	download from the console

Check that both answer:

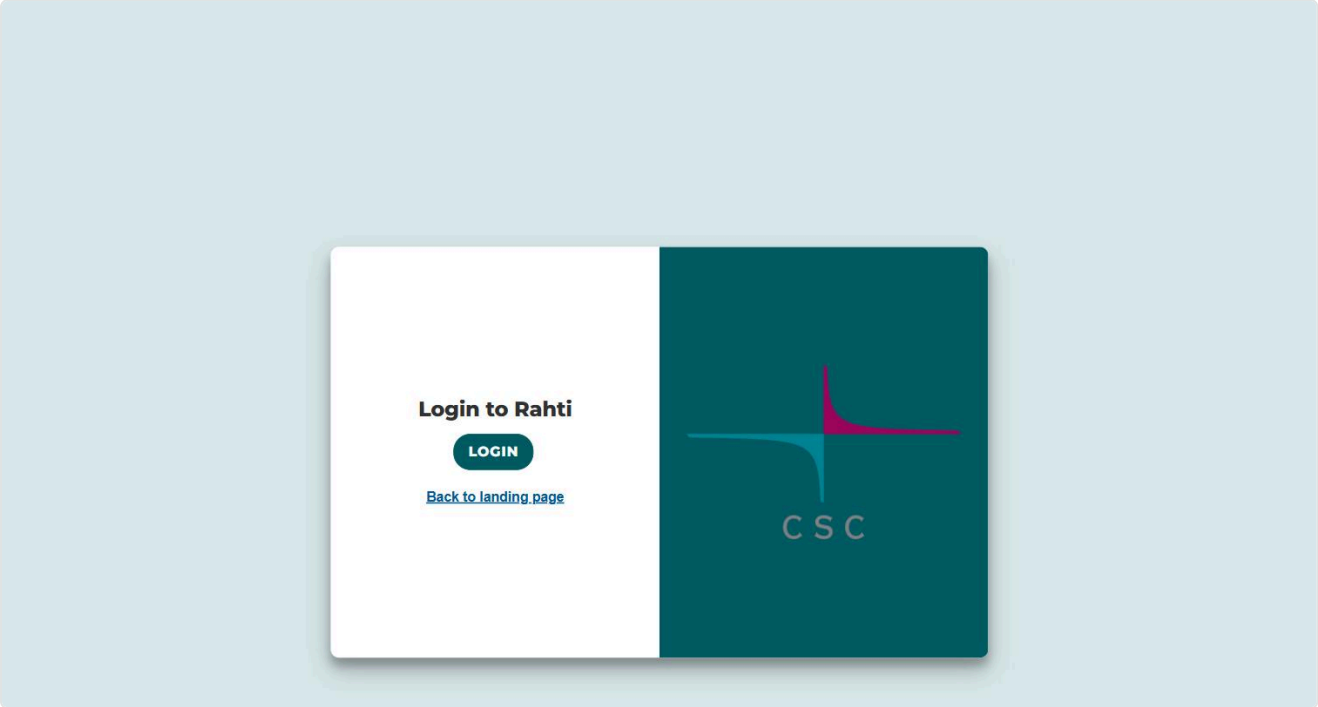
```
docker --version     # or: podman --version
oc version --client
```

> Podman runs containers without root privileges, which is closer to how Rahti runs them. The commands are the same: replace `docker` with `podman`.

> The same computing project can also be used with cPouta or Roihu — Rahti is simply added to its list of available services.

Logging in from the browser

Go to https://console.rahti.csc.fi and click **LOGIN**.



Choose an authentication method: **Haka** (higher education institutions), **Virtu** (government), or **CSC**. All of them require multi-factor authentication (MFA) — mandatory from November 25, 2025. MFA cannot be bypassed with a script or an AI agent.

> **"User not found" after logging in?** The account exists, but the computing project hasn't been granted Rahti access yet, or the sync is still in progress. Check MyCSC.

Creating a project

In Rahti, everything runs inside a **project** (a Kubernetes *namespace*). A project has its own network, its own secrets, and its own access list.

In the browser: left menu → *Home* → *Projects* → **Create Project**.

From the command line — note the `csc_project` description, which links the Rahti project to the computing project and thus to billing:

```
oc new-project my-project --description="csc_project: 2001234"
```

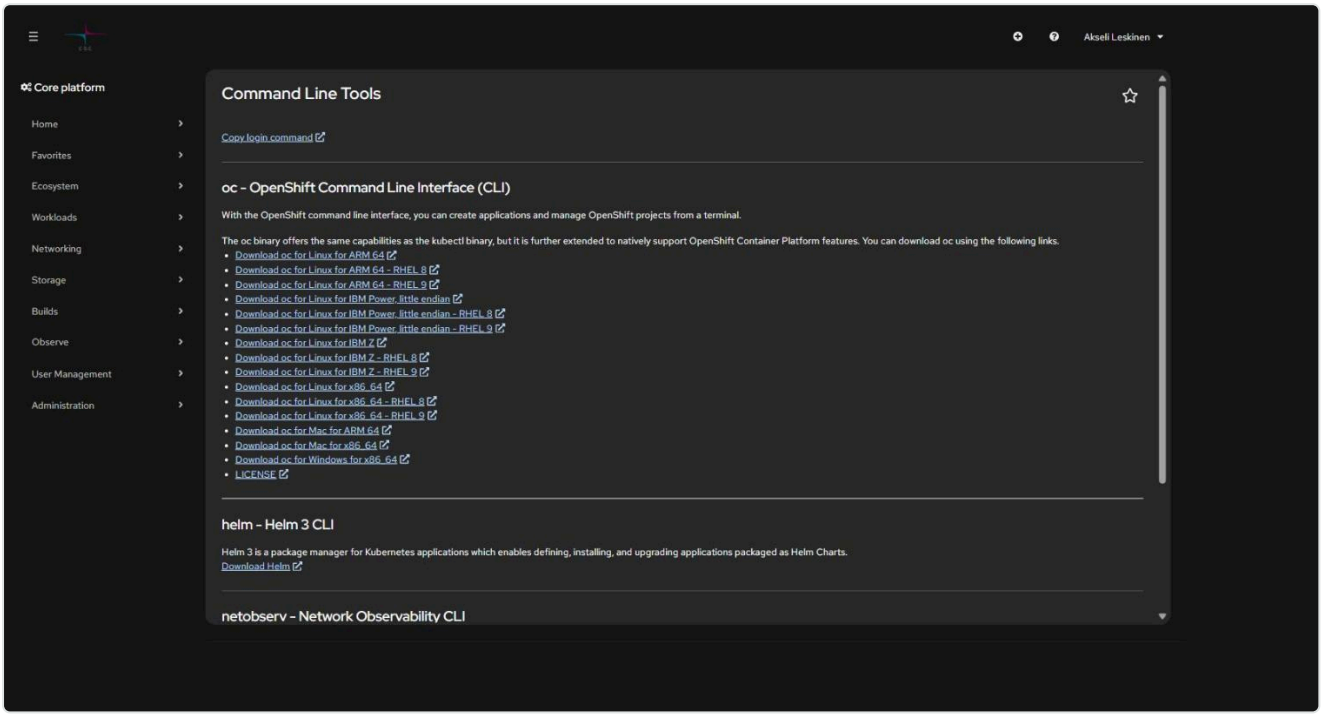
By default, all members of the computing project get admin rights on the created Rahti project. Individual users can be added in the console under *User Management* → *RoleBindings*.

> **Naming:** the project name appears in the app's default address in the form `<app>-<project>.2.rahtiapp.fi`, so pick something short and clear.

The `oc` command-line tool

`oc` is the OpenShift CLI. It can do everything `kubectl` can, plus OKD-specific things (routes, image streams, build configs).

Download the binary directly from the console: **? menu → Command Line Tools**.



On Windows, the easiest way is a package manager:

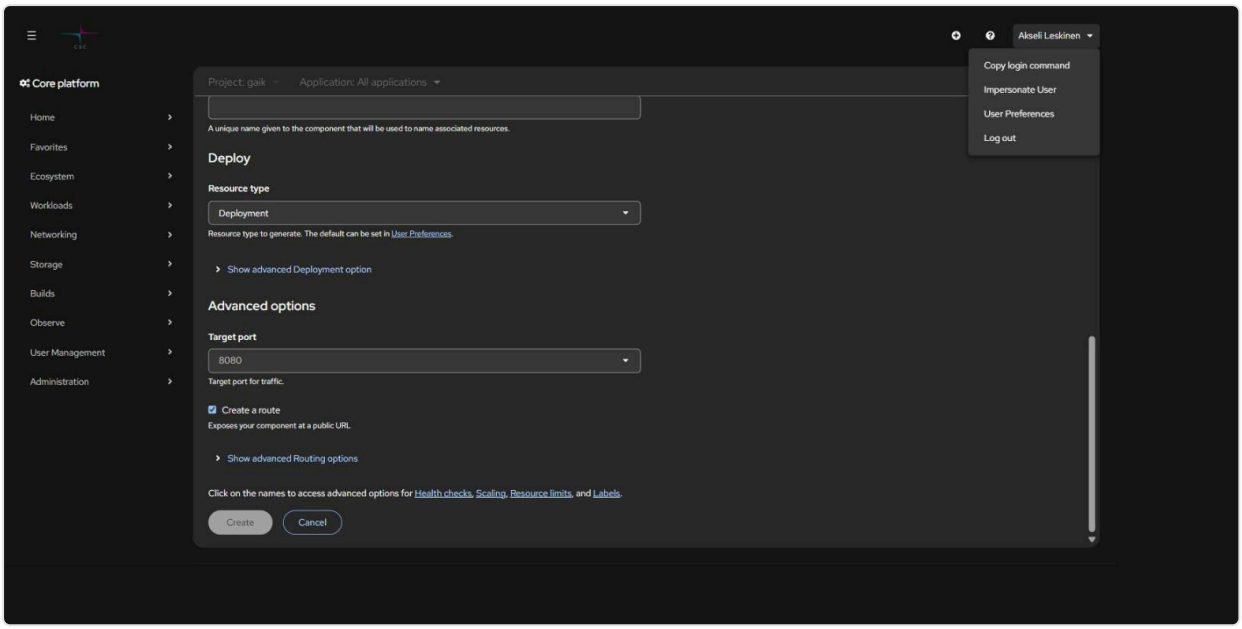
```
# Scoop
scoop install openshift-cli

# or download the zip from the page above and extract oc.exe into a folder on your PATH
```

Check the installation:

```
oc version --client
```

Logging in from the command line

1. In the console, click your name in the top-right corner → **Copy login command**



2. On the page that opens, click **Display Token**

3. Copy the command and run it in your terminal:

```
oc login https://api.2.rahti.csc.fi:6443 --token=sha256~<your-token>
```

Login is machine-specific and shared across all terminal windows. **A personal token expires after about a day.**

Check who you are and which project you're in:

```
oc whoami                  # username, or system:serviceaccount:<ns>:<name>
oc project                 # current project
oc projects                # all projects you have access to
oc project my-project      # switch project
```

Service account for long-term use

A personal token expires every day, which is inconvenient for CI pipelines, deploy scripts, and AI agents. The solution is a **service account**: it has no MFA, and you can choose the lifetime of its token yourself.

```
# 1. Create an account in the project
oc create serviceaccount deployer-bot -n my-project

# 2. Grant it a role (view = read, edit = deploy, admin = also manage permissions)
oc adm policy add-role-to-user edit \
  system:serviceaccount:my-project:deployer-bot -n my-project

# 3. Request a token — a year, in this case
oc create token deployer-bot -n my-project --duration=8760h
```

Store the token in a secrets manager or a gitignored env file — **never commit it**. Usage:

```
oc login https://api.2.rahti.csc.fi:6443 --token=<service-account-token>
```

Revoking it is just as quick:

```
oc adm policy remove-role-from-user edit \
  system:serviceaccount:my-project:deployer-bot -n my-project
oc delete serviceaccount deployer-bot -n my-project
```

> Use a different account for each purpose. For just pushing images, the `system:image-pusher` role is enough — you don't need `edit`.
>
> This repository's csc-rahti skill includes PowerShell scripts (`Initialize-RahtiCredential.ps1` , `Connect-Rahti.ps1`) that automate the whole lifecycle and store the token outside the repository.

Quotas and resource limits

A quota is **per computing project and shared across all of its Rahti projects** — not per app. Default quota for a new computing project:

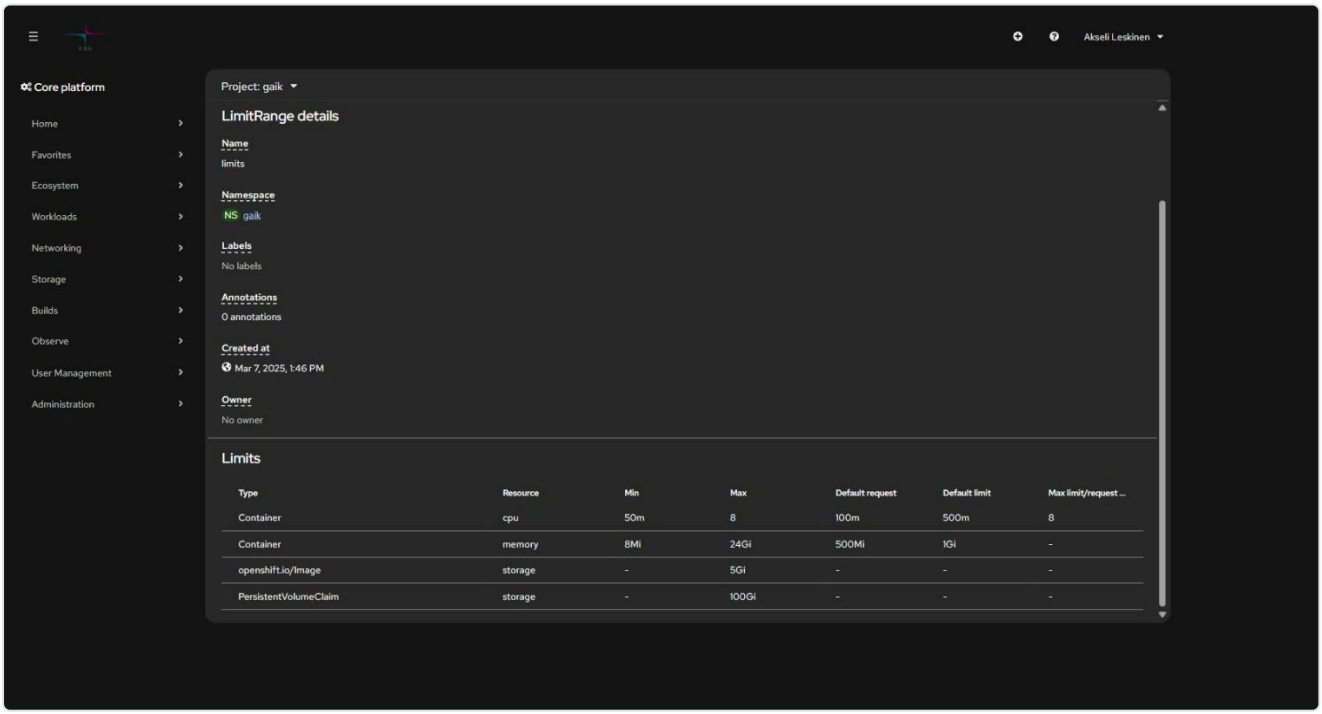
Resource	Default
vCPUs	4
Memory	16 GiB
Storage (PVC)	100 GiB
Ephemeral storage	5 GiB
Number of image streams	20
Concurrent pods	100
Number of PVCs	20

```
oc describe AppliedClusterResourceQuotas   # usage for the whole computing project
oc describe limitranges -n <namespace>     # limits for an individual container
```

The defaults land on the pod even if the Deployment doesn't request anything:

```
oc get pod <pod> -o jsonpath='{.spec.containers[0].resources}'
# {"limits":{"cpu":"500m","memory":"1Gi"},"requests":{"cpu":"100m","memory":"500Mi"}}
```

LimitRange determines what an individual container is allowed to request — and what it gets if you don't request anything:



Type	CPU	Memory
`requests` (reserved)	100m	500Mi
`limits` (ceiling)	500m	1Gi

In addition: the `limits` / `requests` ratio can be at most 5, a single image can be at most 5 GiB, and a single PVC at most 100 GiB. In the screenshot above, the project has been granted limits higher than the default — **always check your own project's actual values**, don't assume.

Additional quota is requested case by case from the CSC service desk.

Security restrictions

Rahti is a shared, multi-tenant environment, so containers are restricted:

- **Root is not allowed.** A container that requires `USER root` will not start.
- **Random UID.** A container gets a project-specific UID at startup. This guide's test app got `1006240000`, even though its Dockerfile says `USER 1001`. So don't write a `runAsUser` value in the manifest, and don't assume a UID. **The group is always `0`** — that's what the write-permission solution relies on.
- **Restricted-v2 policy:** `allowPrivilegeEscalation` must not be `true`, all capabilities must be dropped (`capabilities.drop: ALL`, with the exception of `NET_BIND_SERVICE`), and `seccompProfile` must be either empty or `RuntimeDefault`.
- **Privileged mode is blocked.**

The practical consequence for your Dockerfile: make writable directories writable by group 0. See 9. Troubleshooting and the skill's Dockerfile examples.

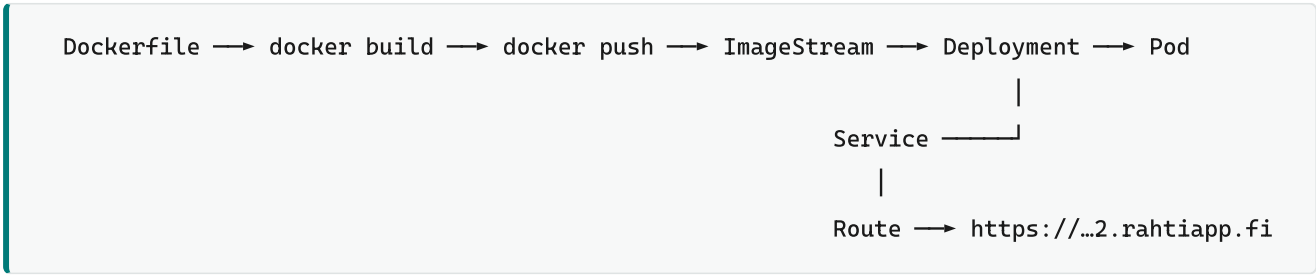
Next: 2. Deploying an application →

2. Deploying an application with Docker

> Building the image, pushing it to a registry, and starting the app on Rahti — both from the browser and from the command line.

Contents

Overview



Four objects are almost always enough:

Object	Role
ImageStream	Rahti's internal reference to the image and its tags
Deployment	Keeps the desired number of pods running, handles rolling updates
Service	A stable internal address for the pods (DNS name + load balancing)
Route	The public HTTPS address, pointing to the Service

Fastest end-to-end path

This script has been run through as-is on Rahti 2 with the example app `examples/hello-rahti`. If you just want to see something working first, clone the repo and run this:

```
cd examples/hello-rahti
oc project <project>

oc create imagestream hello-rahti                       # BEFORE the push
docker build --platform linux/amd64 -t hello-rahti .
docker login -u unused -p $(oc whoami -t) image-registry.apps.2.rahti.csc.fi
docker tag  hello-rahti image-registry.apps.2.rahti.csc.fi/<project>/hello-rahti:latest
docker push image-registry.apps.2.rahti.csc.fi/<project>/hello-rahti:latest

oc new-app --image-stream=hello-rahti                   # creates the Deployment
oc expose deployment/hello-rahti --port=8080            # creates the Service - this does
NOT happen automatically
oc create route edge hello-rahti --service=hello-rahti --insecure-policy=Redirect

curl -s https://$(oc get route hello-rahti -o jsonpath='{.spec.host}')/health
```

> `oc new-app` **doesn't create a Service.** It creates only a Deployment, and creating the route then fails with the error "*you need to provide a route port via --port when exposing a non-existent service*". `oc expose deployment/<name> --port=<port>` creates the Service with the right selector. This is one of the most commonly hit snags in this guide — and it's missing from most tutorials you'll find online.

The rest of this chapter explains what each step does, and how to do it in a more durable way.

1. Build the image

```
# Basic command, run in the project root
docker build -t myapp .

# Without cache, if you suspect a stale layer
docker build --no-cache -t myapp .
```

Test locally before shipping to the cloud:

```
docker run -p 3000:3000 myapp
docker run --env-file .env -p 3000:3000 myapp
```

> **A Rahti-compatible Dockerfile:** don't run as root, don't hardcode a UID, and make writable directories writable by group 0. Ready-made templates (Next.js, Python/FastAPI, static site) are in the skill's dockerfile-examples.md.

> **Processor architecture:** Rahti runs `linux/amd64`. An image built on an Apple M-series machine defaults to `arm64` and won't start on Rahti. Use `docker build --platform linux/amd64 …`.

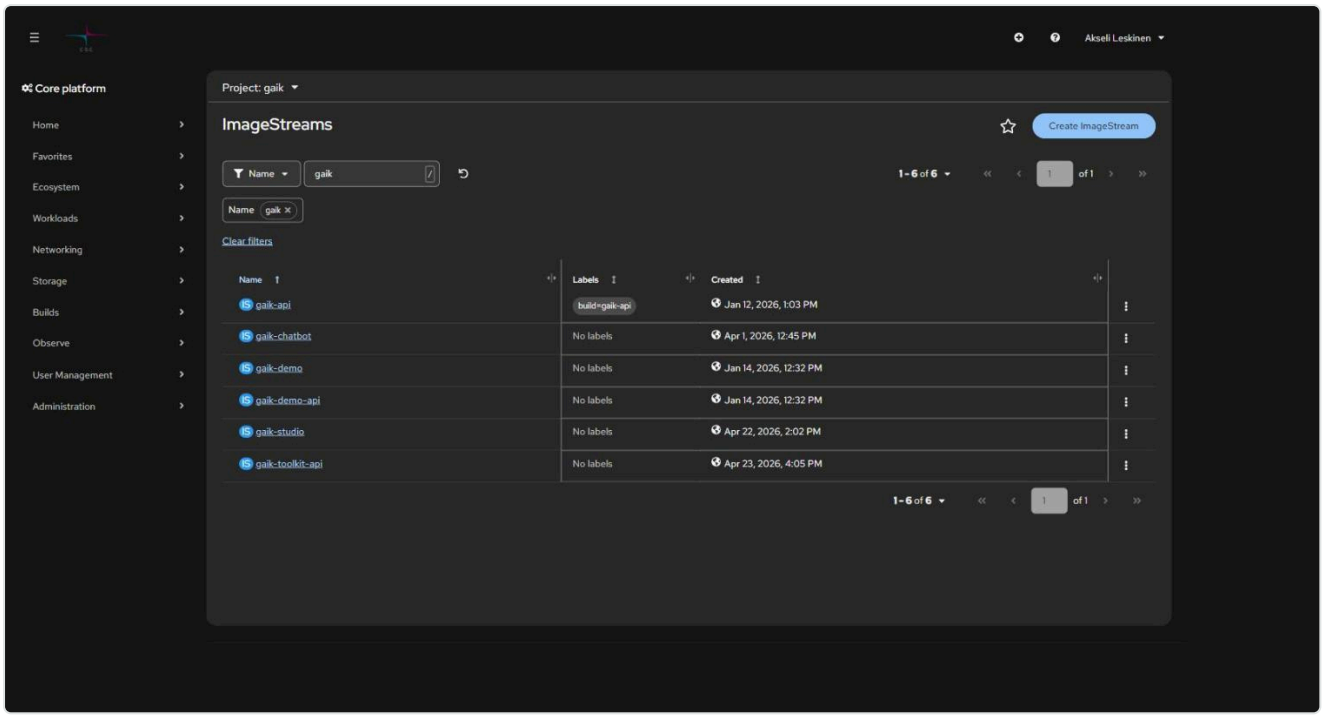
2. Choose a registry

Registry	When
Rahti's internal registry `image-registry.apps.2.rahti.csc.fi`	Fast dev cycle within a single project. The default choice.
Satama `satama.csc.fi`	The image is shared across projects or clusters, or you need vulnerability scanning, signing, and long-lived robot accounts.
Docker Hub	A public image, or no CSC project involved at all.

Option A: Rahti's internal registry

```
# 1. Create the ImageStream FIRST — without it, the push fails with an HTTP 500 error
oc get is -n <project>                          # list existing ones
oc create imagestream myapp -n <project>

# 2. Log in to the registry (requires a valid oc session)
docker login -u unused -p $(oc whoami -t) image-registry.apps.2.rahti.csc.fi

# 3. Tag
docker tag myapp \
  image-registry.apps.2.rahti.csc.fi/<project>/myapp:latest

# 4. Push
docker push image-registry.apps.2.rahti.csc.fi/<project>/myapp:latest
```

If the ImageStream is missing, the push fails with `unexpected status from HEAD request … : 500`, which doesn't mention ImageStream anywhere. So remember the order.



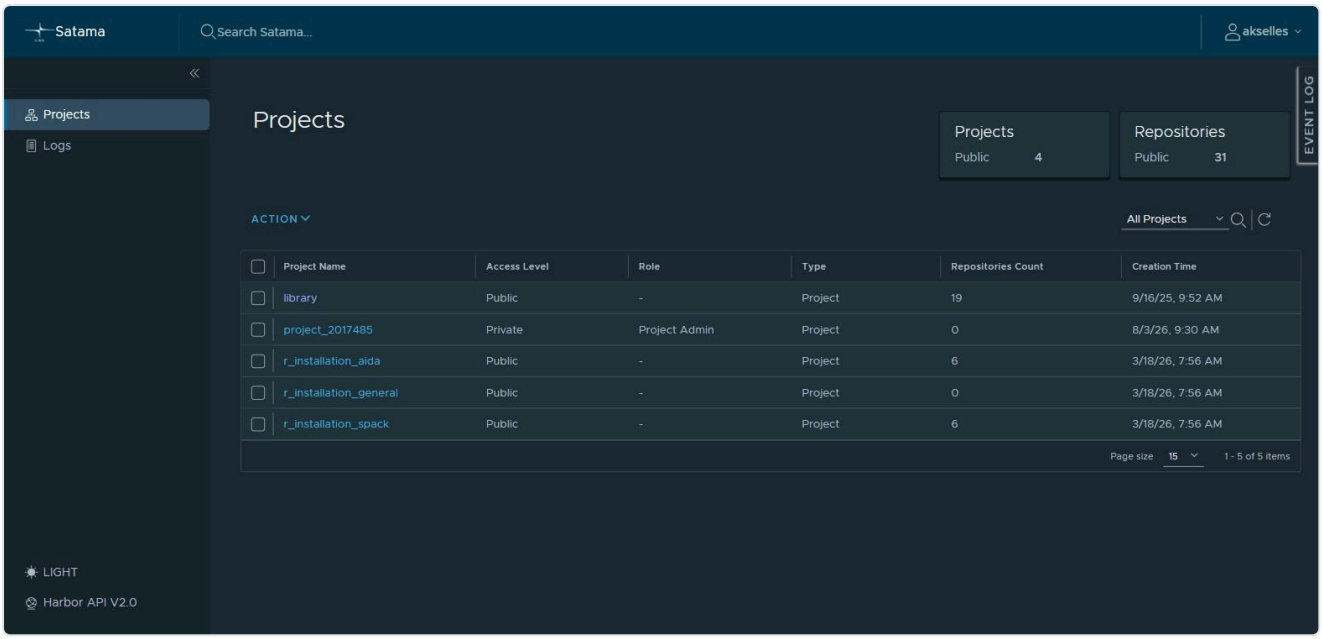
> **Shortcut for the first time:** push the image to Docker Hub first and deploy it on Rahti from there. Rahti creates the ImageStream automatically, after which you can switch to the internal registry.

For a CI pipeline, it's worth creating a dedicated account with push-only rights:

```
oc create serviceaccount pusher -n <project>
oc policy add-role-to-user system:image-pusher -z pusher -n <project>
docker login -u unused -p $(oc create token pusher --duration=8760h) \
  image-registry.apps.2.rahti.csc.fi
```

Option B: Satama (CSC's Harbor)

Satama is CSC's own container registry, a service separate from Rahti. You log in with a **CLI secret** (Satama UI → your username → *User Profile* → *CLI Secret*), not your MyCSC password.



In this view, `library` and `r_installation_*` are CSC's own public projects, which you can pull images from without logging in. `project_<number>` is your own computing project.

```
docker login satama.csc.fi -u <csc-username>      # paste the CLI secret
docker tag  myapp:1.0 satama.csc.fi/<satama-project>/myapp:1.0
docker push satama.csc.fi/<satama-project>/myapp:1.0
```

Pulling from a private Satama project requires a pull secret:

```
oc create secret docker-registry satama-pull \
  --docker-server=satama.csc.fi \
  --docker-username='robot$<project>+<name>' \
  --docker-password="$SATAMA_ROBOT_SECRET" -n <project>
oc secrets link default satama-pull --for=pull -n <project>
```

Public Satama projects can be pulled without a secret. For robot accounts, scanning, tag immutability, and quotas, see satama-registry.md.

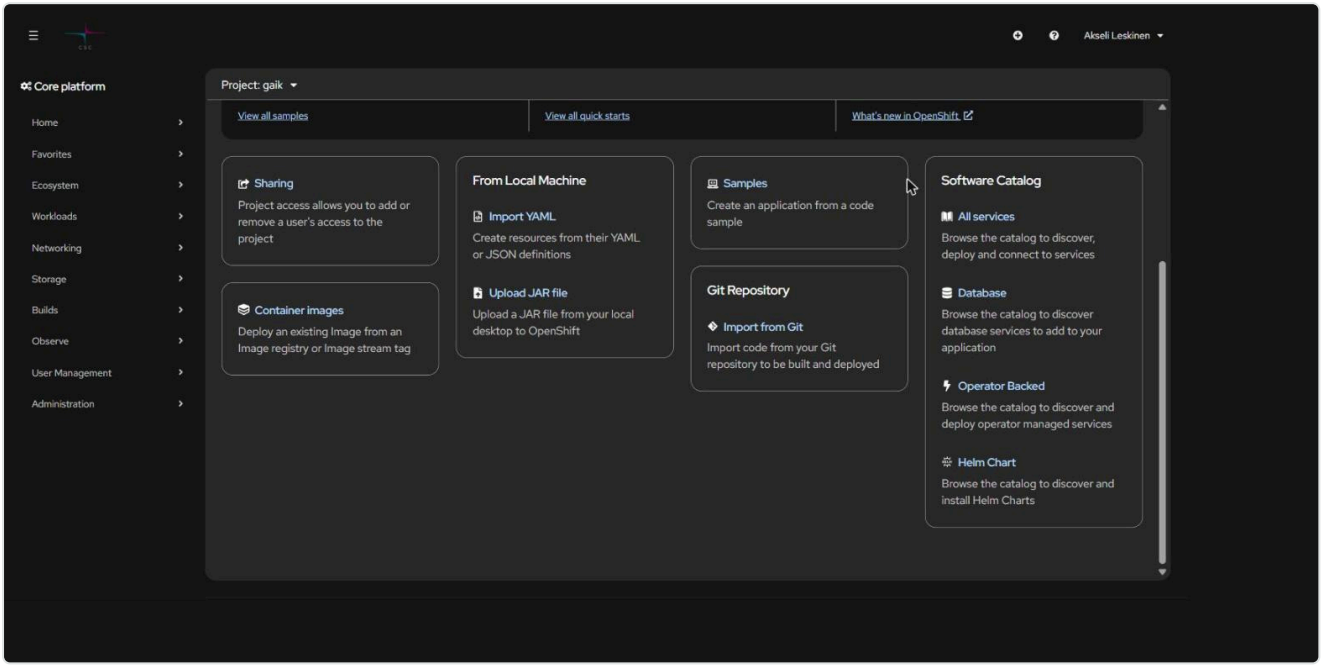
Option C: Docker Hub

```
docker tag myapp <dockerhub-username>/myapp:latest
docker push <dockerhub-username>/myapp:latest
```

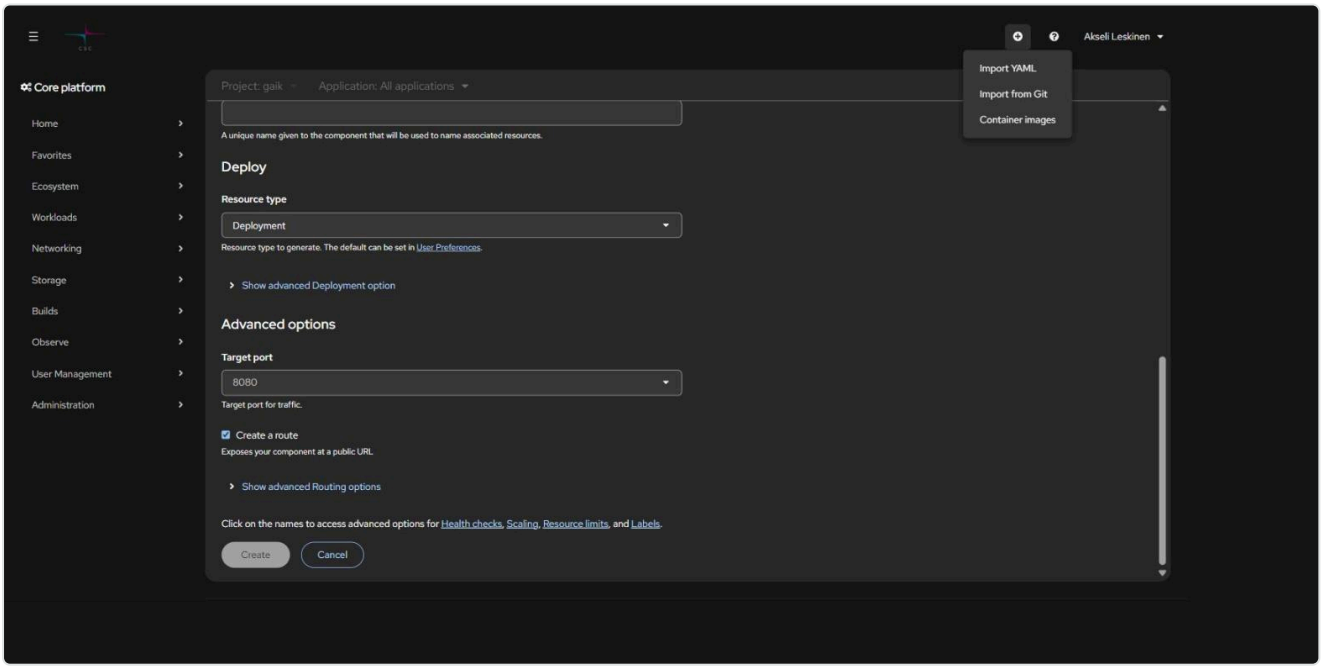
A private Docker Hub image needs an *image pull secret* in Rahti (registry `docker.io`, username, and password/token). A public image needs nothing.

3. Deploy from the browser

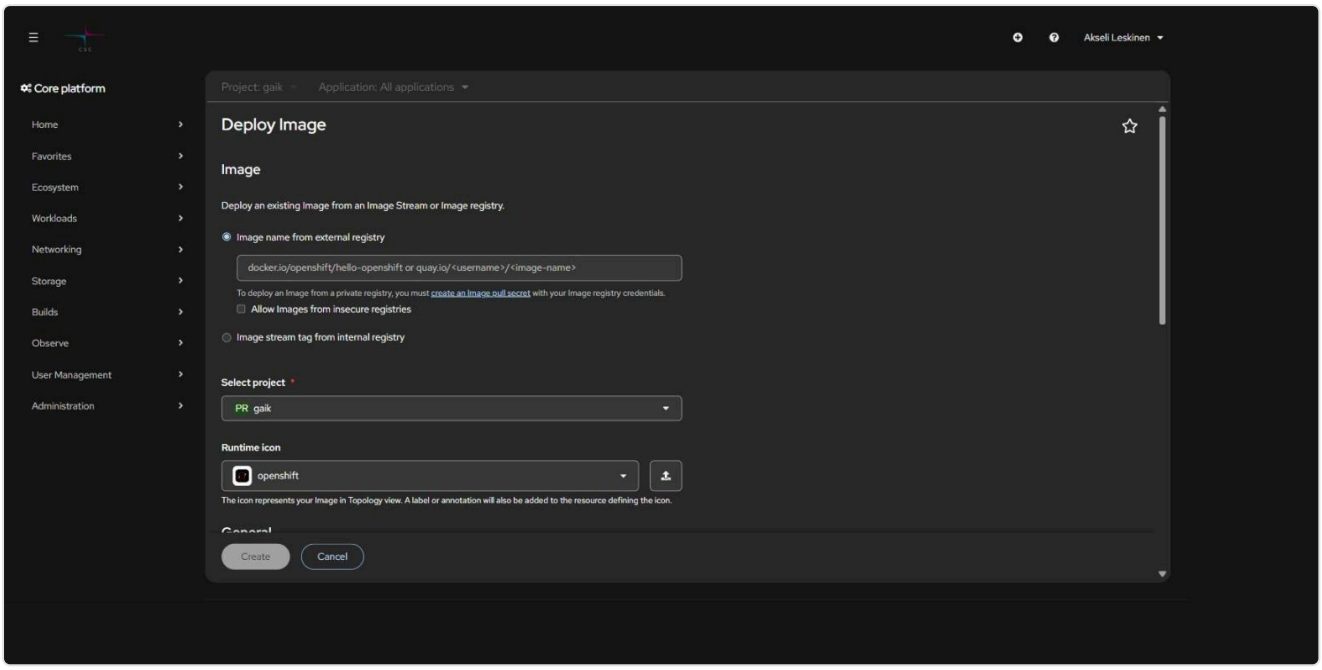
The console's left menu is now a unified **Core platform** navigation; there's no longer a separate Developer/Administrator view switch. A new app is added either from the **+** quick menu at the top or from the project's **+Add** page.



The quickest route: **+** (top) → *Container images*.

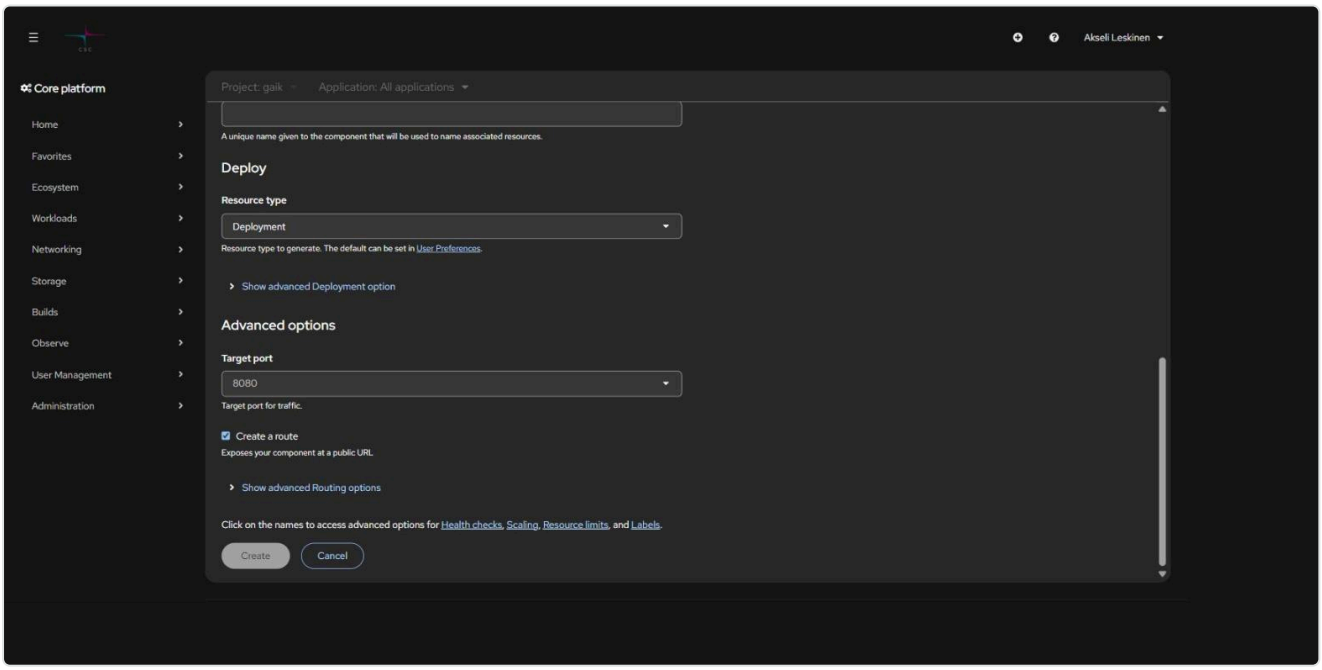


Deploy Image form:



- **Image name from external registry** — enter e.g. `user/app:latest` (Docker Hub) or `satama.csc.fi/project/app:1.0`
- **Image stream tag from internal registry** — pick a previously pushed image
- **Select project** — the target project
- **Application** — the logical group the component belongs to (shown in the Topology view)
- **Name** — the component's name, from which the Deployment, Service, and Route are derived

Scroll down to the **Advanced options** section:



- **Target port** — the port your app listens on (e.g. 3000 or 8080)
- **Create a route** — creates a public HTTPS address. Leave unchecked if the service should only be internal (e.g. a backend or a database).

Click **Create**. Rahti creates the Deployment, Service, and, if requested, the Route, generates the address `<name>-<project>.2.rahtiapp.fi`, and handles TLS (edge termination, HTTP redirected to HTTPS).

After deploying, the app shows up in the **Topology** view, grouped by application:

4. Deploy from the command line (manifests)

Once you're deploying an app repeatedly, the browser form starts to get in the way: the configuration isn't version-controlled. Recommended layout:

```
<repo>/openshift/
  manifests/            # declarative objects — run `oc apply -f` ONCE
    00-imagestreams.yaml
    10-api-deployment.yaml   11-api-service.yaml
    20-web-deployment.yaml   21-web-service.yaml   22-web-route.yaml
  deploy.sh             # repeatable loop: build → push → rollout
```

- **Once:** `oc apply -f openshift/manifests/` and secrets via `oc create secret generic … --from-literal=…`
- **On every code change:** `./openshift/deploy.sh`

A minimal Deployment + Service + Route:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 1
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      securityContext: {}          # NO runAsUser — the SCC assigns the UID
      containers:
        - name: myapp              # the name must match the image trigger
          image: image-registry.openshift-image-registry.svc:5000/<project>/myapp:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 3000
          resources:
            requests: { cpu: 100m, memory: 256Mi }
            limits:   { cpu: 500m, memory: 512Mi }
          readinessProbe:
            httpGet: { path: /health, port: 3000 }
            initialDelaySeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: myapp
spec:
  selector:
    app: myapp
  ports:
    - name: http
      port: 3000
      targetPort: 3000
---
apiVersion: route.openshift.io/v1
kind: Route
```

```
metadata:
  name: myapp
spec:
  host: myapp.2.rahtiapp.fi
  to:
    kind: Service
    name: myapp
  port:
    targetPort: http
  tls:
    termination: edge
    insecureEdgeTerminationPolicy: Redirect
```

Note that inside the cluster, the image is referenced by `image-registry.openshift-image-registry.svc:5000/...`, not by the public `image-registry.apps.2.rahti.csc.fi` name.

Two things worth knowing before you go looking for a bug in the wrong place:

- **You can leave out the `resources` block entirely**, and the LimitRange supplies the defaults (100m/500m CPU, 500Mi/1Gi memory). They don't show up in the Deployment's spec, only on the pod: `oc get pod <pod> -o jsonpath='{.spec.containers[0].resources}'`.
- **`securityContext: {}` is intentional.** The `restricted-v2` SCC assigns the pod a random UID (e.g. `1006240000`) in group 0. If you write a `runAsUser` value, the pod gets rejected.

Automatic restart on a new image

A plain `docker push` doesn't restart the pod. Add an **image trigger** to the Deployment, and the new `:latest` is picked up automatically:

```
metadata:
  annotations:
    image.openshift.io/triggers: |
      [{"from":{"kind":"ImageStreamTag","name":"myapp:latest"},
        "fieldPath":"spec.template.spec.containers[?(@.name==\"myapp\")].image"}]
```

The trigger rewrites the `.image` field to an `@sha256` digest, so the live object won't show `:latest`. That's correct behavior, not a bug.

The same from the command line:

```
oc set triggers deployment/myapp \
  --from-image=myapp:latest -c myapp -n <project>
```

If the deployment doesn't update after a push, see 9. Troubleshooting.

A repeatable deploy script

A `deploy.sh` that works in practice does four things:

```
#!/usr/bin/env bash
set -euo pipefail

NS=my-project
APP=myapp
REG=image-registry.apps.2.rahti.csc.fi
TAG=$(date +%Y%m%d-%H%M%S)

docker build --platform linux/amd64 -t "$APP" .
docker tag "$APP" "$REG/$NS/$APP:latest"
docker tag "$APP" "$REG/$NS/$APP:$TAG"
docker push "$REG/$NS/$APP:latest"
docker push "$REG/$NS/$APP:$TAG"

# Force a rollout even when the digest hasn't changed (e.g. a changed Secret)
oc patch deployment/"$APP" -n "$NS" -p \
  "{\"spec\":{\"template\":{\"metadata\":{\"annotations\":
{\"kubectl.kubernetes.io/restartedAt\":\"$(date -Iseconds)\"}}}}}"
oc rollout status deployment/"$APP" -n "$NS" --timeout=180s
```

> **Secrets don't update themselves.** A changed Secret isn't reflected in a running pod until it restarts — hence the `restartedAt` patch.

Binary build as an alternative

If you don't want to run Docker locally, Rahti can build the image for you from source code:

```
# Once
oc new-build --name=myapp --binary --strategy=docker \
  --to=myapp:latest -n <project>

# On every build, in the project root
oc start-build myapp --from-dir=. --follow -n <project>
```

This is handy for testing. For ongoing use, we recommend GitHub integration: 3. GitHub integration →

3. GitHub integration (BuildConfig and webhooks)

> `git push` → webhook → Rahti builds the image → the app updates. No local Docker, no manual pushes.

Contents

When this is worth it

Approach	Good	Bad
Local docker push (chapter 2)	Fast, full control, works offline	Requires Docker, manual
BuildConfig + webhook (this chapter)	Automatic, no local Docker needed, the build is logged in the cluster	The build consumes the project's quota, slower, errors are debugged from the build logs
GitHub Actions → docker push	Familiar pipeline, tests in the same place	Requires a Rahti token in the CI secrets

Builder Image or your own Dockerfile

Builder Image (S2I) — Rahti packages your source code into a ready-made runtime. No Dockerfile, quick to get started, sufficient for a typical Node.js or Python app.

Your own Dockerfile (Docker strategy) — full control, multi-stage builds, necessary for e.g. a Vite/React frontend. Requires the workaround described below.

> Builder image versions go out of date. Pick an up-to-date UBI-based version from the list; a Node.js 18 builder that's years old no longer gets security updates.

A known error: "URL is valid but cannot be reached"

When you try to create a project directly with *Import from Git* using an SSH URL and the Dockerfile strategy, Rahti shows the error **"URL is valid but cannot be reached"** even though the SSH key is correctly set up in both GitHub and Rahti.

Workaround:

1. First create the app with a **Builder Image**. The red "URL is valid but cannot be reached" message stays under the field, but it's just a warning: the **Create** button works and the project is created normally. The error only concerns validation of the Dockerfile strategy.
2. Then edit the BuildConfig (*Builds → BuildConfigs → Actions → Edit BuildConfig* or the YAML directly) to the Docker strategy:

```
strategy:
  type: Docker
  dockerStrategy:
    dockerfilePath: Dockerfile
```

Alternatively, create the BuildConfig entirely from the command line, so the form's validation bug doesn't get in the way (see step 2).

Step 1: SSH key for GitHub

A public repository just needs an HTTPS URL — no key required. For a private one:

```
# 1. Create a key pair (empty passphrase — Rahti has no way to prompt for one)
ssh-keygen -t ed25519 -C "rahti-deploy" -f ./rahti_github_key

# 2. Add the public key to GitHub:
#    repo → Settings → Deploy keys → Add deploy key → paste rahti_github_key.pub
#    Leave "Allow write access" off — the build only needs read access.

# 3. Import the private key into Rahti as a secret
oc create secret generic github-ssh-key \
  --type=kubernetes.io/ssh-auth \
  --from-file=ssh-privatekey=./rahti_github_key -n <project>

oc secrets link builder github-ssh-key -n <project>

# 4. Remove the private key from disk
rm rahti_github_key
```

In the browser, the same thing is done in the *Import from Git* form under **Show advanced Git options** → **Source Secret** → **Create new Secret** (type *SSH key*). Paste the entire key, including the `BEGIN` and `END` lines.

Step 2: BuildConfig

```
apiVersion: build.openshift.io/v1
kind: BuildConfig
metadata:
  name: myapp
spec:
  source:
    type: Git
    git:
      uri: git@github.com:<org>/<repo>.git
      ref: main                      # see Pitfalls!
    sourceSecret:
      name: github-ssh-key
  strategy:
    type: Docker
    dockerStrategy:
      dockerfilePath: Dockerfile
  output:
    to:
      kind: ImageStreamTag
      name: myapp:latest
  triggers:
    - type: ConfigChange
    - type: GitHub
      github:
        secretReference:
          name: github-webhook-secret
---
apiVersion: image.openshift.io/v1
kind: ImageStream
metadata:
  name: myapp
```

```
# Webhook secret first
oc create secret generic github-webhook-secret \
  --from-literal=WebHookSecretKey=$(openssl rand -hex 20) -n <project>

oc apply -f buildconfig.yaml -n <project>
```

Step 3: Webhook

1. **Get the URL from Rahti:** *Builds → BuildConfigs → <name> → Webhooks →* ***Copy URL with Secret***

   From the command line:

   ```
   oc describe bc/myapp -n <project> | grep -A2 "Webhook"
   ```

   Format:

   ```
   https://api.2.rahti.csc.fi:6443/apis/build.openshift.io/v1/namespaces/<project>/buildconfigs/<bc>/webhooks/<secret>/github
   ```

   > The generic (non-GitHub) endpoint ends in `/generic`. Choose the type based on where your code lives: GitHub, GitLab, Bitbucket, or Generic.

2. **Add the webhook in GitHub:** repo → *Settings → Webhooks → Add webhook*
   - **Payload URL:** the copied address
   - **Content type:** `application/json`
   - **Secret:** the same secret as in the URL
   - **SSL verification:** Enable
   - **Which events:** *Just the push event*
   - **Active:** ✓
3. **Test it:** make a change, push it, and check whether a build starts.

Tracking builds

```
oc get builds -n <project>                  # all builds
oc logs build/<build-name> -n <project>     # logs for one build
oc logs -f bc/myapp -n <project>            # follow the latest build
oc start-build myapp -n <project>           # start manually
```

Pitfalls

Branch name. Rahti's BuildConfig defaults to the `master` branch, GitHub's default is `main`. If the `ref` field isn't set, pushes to `main` are silently ignored with no error message. Set `spec.source.git.ref: main` — or in the browser, *Show advanced Git options → Git reference*.

A failed build is a silent failure. A crashed build doesn't break the running app: no new image is produced, and the pod just keeps running the old code. The pod shows `Ready 1/1` and the route returns 200 — but the code change doesn't show up. Always check `oc get builds` when "the deploy didn't go through."

The build consumes quota. The build pod reserves CPU and memory from the same computing project quota as your apps. If the quota is full, the build stays `Pending`.

Build pods pile up. Every run leaves behind a `<app>-<n>-build` pod. `Completed` / `Succeeded` is normal, `Failed` is a genuine finding. Clean up old ones as needed:

```
oc delete pods -n <project> --field-selector=status.phase==Succeeded
```

Pulling a private image. If the BuildConfig pushes to Satama or pulls a base image from a private registry, link the pull secret to the `builder` service account too:

```
oc secrets link builder <pull-secret> -n <project>
```

4. Routes, networking, and URLs

> How an app gets a public address, how services talk to each other inside the cluster, and what to do when a long request cuts off after 30 seconds.

Contents

Three layers: Pod, Service, Route

```
Internet ──► Route (https://app.2.rahtiapp.fi)
                │  TLS is terminated here (edge)
                ▼
             Service (app:3000)        ← stable DNS name and load balancing
                │
                ▼
             Pod, Pod, Pod             ← IPs change, pods die and get created
```

- **A pod's IP changes** every time the pod restarts. Never rely on it for anything.
- **Service** is a stable name and IP that routes traffic based on a label selector.
- **Route** is the only thing visible to the internet. Without a Route, a service is only reachable from inside the cluster — which is the **right** choice for a backend or a database.

By default, a pod can only talk to pods in its own project (NetworkPolicy).

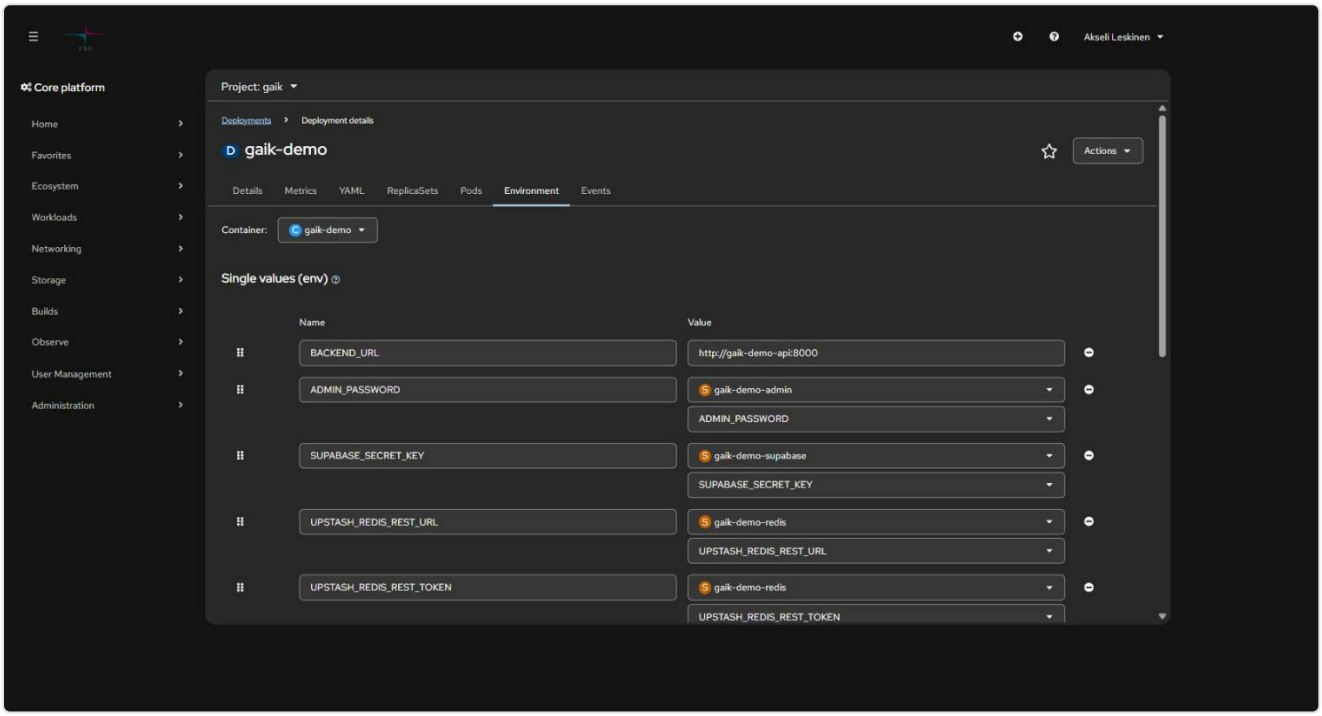
Internal traffic: Service DNS

Every Service gets a DNS name:

```
<service>.<project>.svc.cluster.local     # full name
<service>.<project>                       # from a different project
<service>                                 # from within the same project
```

In practice, the frontend calls the backend like this:

```
oc set env deployment/frontend -n <project> \
  BACKEND_URL=http://backend-api:8000
```



Note `http://`, not `https://` — TLS is already terminated at the Route, and the internal network carries traffic inside the cluster. The port is the **Service** port, not the Route's.

Testing the connection from a pod:

```
oc exec -it deployment/frontend -n <project> -- sh
# inside the pod:
wget -qO- http://backend-api:8000/health
```

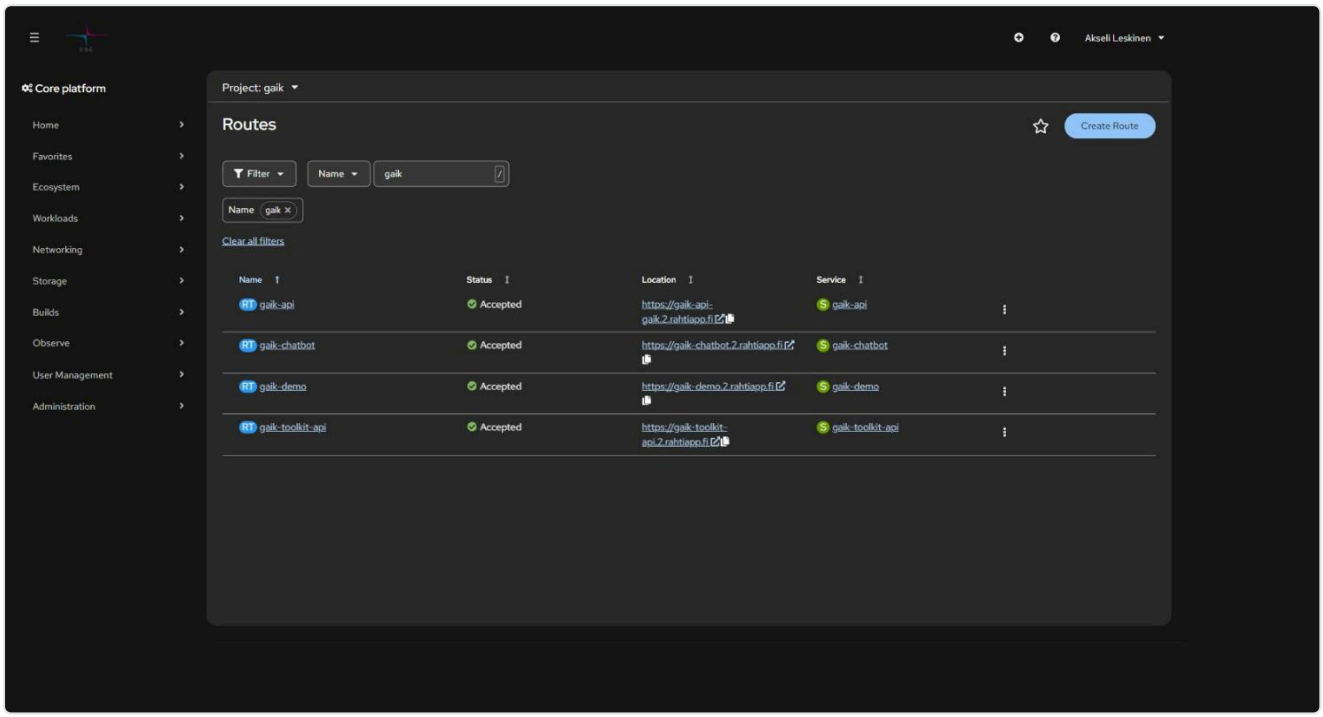
> **Best practice:** don't give the backend a Route at all. That way it can't be called from the internet, and you don't have to build authentication twice.

More patterns: 6. Frontend and backend.

Public address: Route

When you create an app from the console and check *Create a route*, Rahti generates an address in the form:

```
https://<component-name>-<project>.2.rahtiapp.fi
```



In the screenshot, `gaik-api` uses the generated name (`gaik-api-gaik.2.rahtiapp.fi`) while the others use a custom one (`gaik-demo.2.rahtiapp.fi`). Both work side by side.

```
oc get routes -n <project>
oc get route <name> -n <project> -o jsonpath='{.spec.host}'
```

Creating your own URL

The address prefix is **unique across the entire Rahti environment**, and the suffix must be `.2.rahtiapp.fi` . A route is its own object, so you get a new address by creating a new Route — you don't need to delete the old one.

```
# new-route.yaml
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: <route-name>
  namespace: <project>
  labels:
    app: <app-name>
  annotations:
    openshift.io/host.generated: "false"
spec:
  host: <desired-name>.2.rahtiapp.fi
  to:
    kind: Service
    name: <service-name>
    weight: 100
  port:
    targetPort: <port-name-or-number>
  tls:
    termination: edge
    insecureEdgeTerminationPolicy: Redirect
  wildcardPolicy: None
```

```
oc project                      # confirm the project
oc apply -f new-route.yaml
curl -I https://<desired-name>.2.rahtiapp.fi
```

Example. An app called `learning-assistant`, whose Service is `learning-assistant` on port 3000:

```
spec:
  host: learning-assistant.2.rahtiapp.fi
  to:
    kind: Service
    name: learning-assistant
  port:
    targetPort: 3000-tcp
```

> `targetPort` refers to the **Service**'s port. For services created from the console, the port name is typically `<number>-tcp`. Check with: `oc get svc <name> -o yaml`.

The same thing with a single command, no YAML file needed:

```
oc create route edge <route-name> \
  --service=<service> --hostname=<name>.2.rahtiapp.fi \
  --insecure-policy=Redirect -n <project>
```

Switching routes without downtime

Multiple Routes can point to the same Service. Do the migration like this:

1. Create a new Route with the new hostname
2. Test it
3. Update links and any CORS settings
4. Remove the old one only once nothing uses it anymore

```
oc delete route <old-route> -n <project>
```

TLS termination

Mode	What happens	When
edge	The router terminates TLS and forwards HTTP to the pod	Default. The app doesn't need to know about certificates.
passthrough	The router doesn't terminate anything; the pod handles TLS	End-to-end encryption, your own certificate in the pod
reencrypt	The router terminates and re-encrypts to the pod	The internal network must also be encrypted, but Rahti's certificate is used externally

For `*.2.rahtiapp.fi` addresses, Rahti's wildcard certificate is enough — you don't need to get your own. **A Route without a `tls` block is served as plain HTTP**, so edge termination has to be requested explicitly.

504 Gateway Time-out on long requests

The Route's HAProxy timeout defaults to **30 seconds**. A long synchronous request — an LLM call, a large file upload, a heavy report — cuts off with a 504 error, which the app often shows as a generic "Request failed" message.

```
oc annotate route <route> \
  haproxy.router.openshift.io/timeout=120s -n <project> --overwrite
```

Or in the manifest:

```
metadata:
  annotations:
    haproxy.router.openshift.io/timeout: 120s
```

First make sure the request actually takes that long — **a 504 at almost exactly ~30 seconds** is the telltale sign of this issue:

```
curl -o /dev/null -s -w 'HTTP %{http_code} in %{time_total}s\n' https://<address>/slow
```

For genuinely long-running jobs, an asynchronous job plus status polling is a better solution than keeping a connection open for minutes at a time.

IP restriction and other annotations

```
# Allow only the higher-education network
oc annotate route <route> \
  haproxy.router.openshift.io/ip_allowlist='193.166.0.0/16' -n <project>

# HSTS (force HTTPS in the browser)
oc annotate route <route> \
  haproxy.router.openshift.io/hsts_header="max-age=31536000;includeSubDomains;preload" -n
<project>
```

> **Warning:** an invalid allowlist value is silently rejected and **all traffic is allowed**. Values are separated by spaces, not commas, and there must be no trailing whitespace.

Custom domain

You can use your own domain by pointing DNS at Rahti's router:

```
CNAME  <your-domain>  →  router-default.apps.2.rahti.csc.fi
```

If a CNAME isn't possible (a root domain), use an A record with that name's IP — but remember the IP can change. The certificate is your own responsibility; Let's Encrypt automation works with Rahti's cert-manager support. Instructions: CSC: Custom domains.

Egress traffic

All traffic currently leaving Rahti uses the IP address `86.50.229.150`. You'll need it if an external database or API is behind a firewall.

CSC warns that the address can change, and parallel Rahti versions running side by side have different addresses. Don't hardcode it without a plan for updating it — always check the value in the documentation. A dedicated, project-specific egress IP can be requested from the service desk.

5. Environment variables and secrets

> The single most common mistake in Rahti projects is mixing up a build-time variable with a runtime one. It leads either to a broken build or to an API key leaking into the browser.

Contents

Build time vs. runtime

Variable type	When it's set	Where it ends up
`NEXT_PUBLIC_*`, `VITE_*`, `REACT_APP_*`	`docker build`	**Baked into the JS bundle — visible to every visitor of the page**
Everything else	When the pod starts (`oc set env`, Secret)	Server side only

Two consequences:

1. **A secret with the `NEXT_PUBLIC_` prefix is public.** It's in the browser's source code. If an API key has ended up there, it has to be rotated.
2. **A runtime variable that the build needs will break the build.** For example, Next.js's `next build` renders the pages and fails if the configuration requires a key that isn't there. The fix is to give the build an empty placeholder and supply the real value only at runtime:

```
# The build accepts an empty value; the real one comes from the Secret when the pod starts
ARG DATABASE_URL=""
ENV DATABASE_URL=$DATABASE_URL
RUN npm run build
```

Setting runtime values

```
# Individual values
oc set env deployment/app -n <project> \
  NODE_ENV=production BACKEND_URL=http://backend-api:8000

# All of a Secret's keys at once
oc set env deployment/app --from=secret/app-env -n <project>

# From a ConfigMap
oc set env deployment/app --from=configmap/app-config -n <project>

# List current values
oc set env deployment/app -n <project> --list

# Remove (note the trailing dash)
oc set env deployment/app -n <project> OLD_VARIABLE-
```

Every `oc set env` automatically triggers a rolling update.

Secrets (Secret)

Passwords, API keys, and tokens belong in a Secret — not in the Deployment's `env` block, and not in version control.

```
# Recommended: one key at a time, no parsing surprises
oc create secret generic app-env \
  --from-literal=DATABASE_URL='postgresql://user:password@postgres:5432/db' \
  --from-literal=OPENAI_API_KEY='sk-...' \
  -n <project>

# Or from a gitignored file (read Pitfalls first!)
oc create secret generic app-env --from-env-file=.env.local -n <project>

# Wire it into the Deployment
oc set env deployment/app --from=secret/app-env -n <project>
```

A single key for just one specific variable:

```
env:
  - name: DATABASE_URL
    valueFrom:
      secretKeyRef:
        name: app-env
        key: DATABASE_URL
```

Checking the value (worth doing right after creating it):

```
oc get secret app-env -n <project> -o jsonpath='{.data.DATABASE_URL}' | base64 -d
```

Updating and rotating:

```
oc create secret generic app-env \
  --from-literal=OPENAI_API_KEY='sk-new' \
  --dry-run=client -o yaml | oc apply -f -

oc rollout restart deployment/app -n <project>
```

ConfigMap for non-secret configuration

```
oc create configmap app-config \
  --from-literal=LOG_LEVEL=info \
  --from-literal=FEATURE_X=true -n <project>

oc set env deployment/app --from=configmap/app-config -n <project>
```

A ConfigMap's contents are visible to everyone with read access to the project — it's not a secret, just structured configuration.

Pitfalls

`--from-env-file` **preserves trailing comments on a line.** The line

```
MODEL=gpt-5.6-sol   # fastest
```

stores the value as `gpt-5.6-sol # fastest`, and the app breaks in a confusing way. Use `--from-literal` instead, or clean up the file first. Always check with `base64 -d`.

Secrets don't update on the fly. A changed Secret isn't reflected in a running pod. Only `oc rollout restart` brings in the new values.

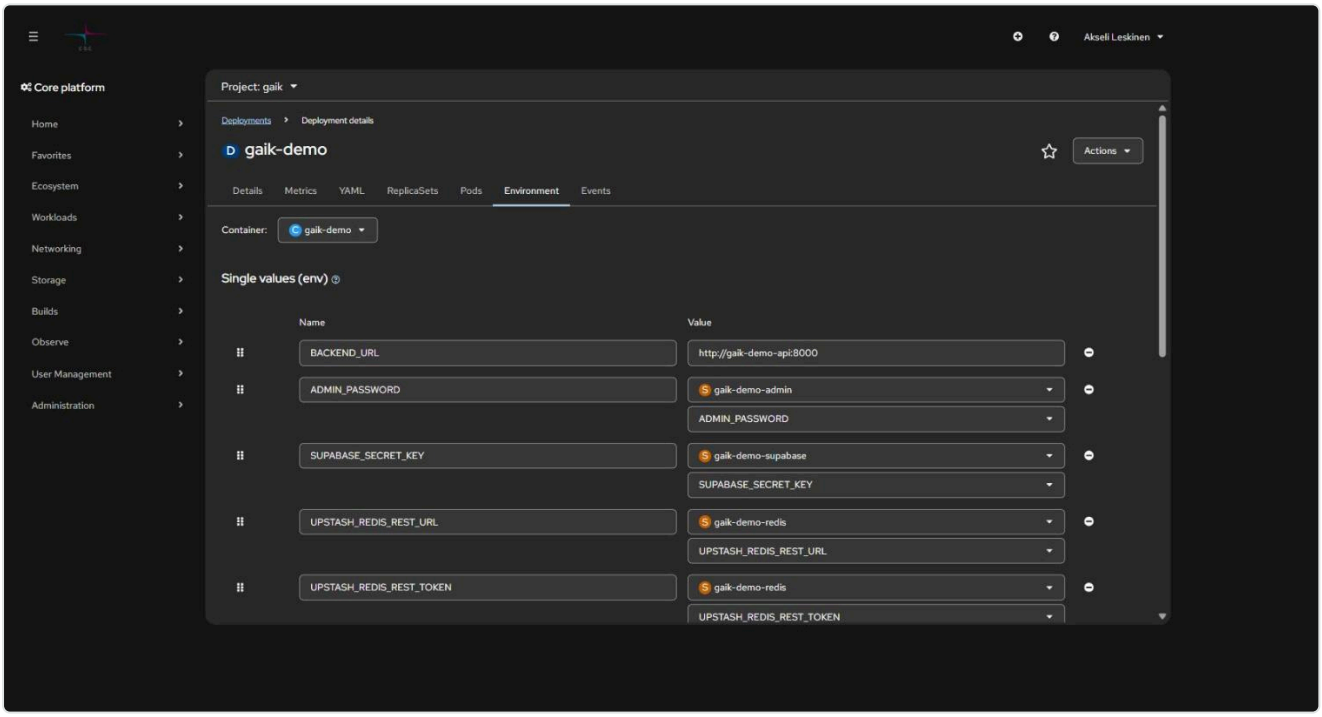
Quoting. A password containing special characters should be wrapped in single quotes so the shell doesn't interpret them: `--from-literal=PW='p@$$w0rd!'`.

`.env` **in version control.** Make sure `.gitignore` includes `.env` and `.env.local`. If a secret has already been committed, removing it from history isn't enough — the key has to be rotated.

A Kubernetes Secret isn't encrypted, only base64-encoded. Anyone with read access to the project can see the value. Restrict project permissions to only those who need them (*User Management → RoleBindings*).

Viewing variables in the console

Workloads → Deployments → <app> → Environment:



In the screenshot, `BACKEND_URL` is a plain value and the rest are pulled from a Secret — the Secret's name and key are visible, the value isn't. This is exactly how secrets should be displayed.

6. Frontend and backend on Rahti

> Three ways to deploy a React (Vite) + Node.js app, and how to wire the services together.

Contents

Choose an architecture

	A: One service	B: Monorepo, one container	C: Separate services
Containers	1	1	2+
CORS needed	no	no	yes (or an internal call)
Scales independently	no	no	yes
Deploy complexity	low	low	medium
Fits	small app, demo	team monorepo	production, different stacks

If you can't decide: start with **A** or **B** and move to **C** once the frontend and backend start evolving at different paces.

Option A: one service, backend serves the frontend

Build the frontend as static files and let the backend serve them. One port, one address, no CORS.

```
cd frontend && npm run build          # produces dist/
cp -r dist ../backend/client/dist     # the backend's static folder
```

```
// backend/index.js
const express = require("express");
const path = require("path");
const app = require("./app");

// Static frontend
app.use(express.static(path.join(__dirname, "client/dist")));

// This MUST come AFTER the API routes — otherwise it also catches /api calls
// app.use works in both Express 4 and 5 (see the note below).
app.use((req, res) => {
  res.sendFile(path.join(__dirname, "client/dist", "index.html"));
});

const PORT = process.env.PORT || 8080;
app.listen(PORT, () => console.log(`Server running on ${PORT}`));
```

> **Express 5 broke `app.get("*")`.** Guides online almost always write the SPA fallback route as `app.get("*", …)`. That works in Express 4 but **throws in Express 5** with `TypeError: Missing parameter name`. Express 5 is the default on npm, so a fresh `npm install express` gets it. The `app.use` above works in both; if you specifically want a route, the Express 5 form is `app.get("/*splat", …)`.

In a monorepo, it's worth automating the copy step in the root `package.json`:

```
{
  "scripts": {
    "build:frontend": "cd frontend && npm install && npm run build",
    "postbuild:frontend": "rm -rf backend/client/dist && cp -r frontend/dist
backend/client/",
    "build": "npm run build:frontend",
    "start": "cd backend && npm start"
  }
}
```

Option B: monorepo in a single container

Same end result as A, but the copying happens in the Dockerfile, so no local build step is needed.

```
project/
├── backend/       (index.js, package.json)
├── frontend/      (src/, package.json)
└── Dockerfile
```

```
# --- Build ---
FROM node:24-alpine AS builder
WORKDIR /app
COPY package*.json ./
COPY frontend/package*.json ./frontend/
COPY backend/package*.json ./backend/
RUN cd frontend && npm ci && cd ../backend && npm ci
COPY . .
RUN cd frontend && npm run build
RUN mkdir -p backend/client && cp -r frontend/dist backend/client/

# --- Runtime ---
FROM node:24-alpine
WORKDIR /app
COPY --from=builder /app/backend ./backend
RUN chown -R 1001:0 /app && chmod -R g+rwX /app
USER 1001
EXPOSE 8080
CMD ["node", "backend/index.js"]
```

> `docker-compose.yml` is handy for local development, but **Rahti does not run compose files**. It reads the Dockerfile and Kubernetes manifests.

Option C: separate services

The frontend and backend are their own Deployments. The key decision: **give the backend a Route or not.**

```
Internet ──► Route ──► frontend-Service ──► frontend-Pod
                                                 │  http://backend-api:8000
                                                 ▼
                                          backend-Service ──► backend-Pod
                                          (no Route = not visible to the internet)
```

Internal call using the service name:

```
oc set env deployment/frontend -n <project> \
  BACKEND_URL=http://backend-api:8000
```

```
// In server-side code (Next.js route handler, Express proxy, …)
const backendUrl = process.env.BACKEND_URL || "http://localhost:8000";
const res = await fetch(`${backendUrl}/api/data`);
```

Frontend Dockerfile (Vite, static serving):

```
FROM node:24-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:24-alpine
WORKDIR /app
COPY --from=builder /app/dist ./dist
RUN npm install -g serve
EXPOSE 8080
CMD ["serve", "-s", "dist", "-p", "8080"]
```

`serve -s` (single) routes every path to `index.html`, which is required for client-side routing (React Router). **If the app is not an SPA**, drop the `-s`: `CMD ["serve", "dist", "-p", "8080"]`.

Remember `.dockerignore`:

```
node_modules
dist
.env
```

> **A call made from the browser can't use the internal name.** `http://backend-api:8000` only works from inside a pod. If the frontend is fully static and calls the API from the browser, the backend needs either its own Route (and CORS) or a server-side proxy on the frontend.

API paths across the models

A and B — same origin, use relative paths:

```
const res = await fetch("/api/endpoint", { method: "POST", body: formData });
```

C — full URL from configuration:

```
// src/config.js
export const BACKEND_URL = import.meta.env.VITE_BACKEND_URL || "http://localhost:8080";
```

Remember: `VITE_*` is baked into the bundle at build time (see 05 Environment variables) — never put secrets there.

CORS

CORS only applies to calls made by the **browser** to a different origin. A `fetch` from the server side (a Next.js route handler, an Express proxy, anything leaving the pod) ignores CORS entirely.

The same parent domain does not mean the same origin. `https://frontend.2.rahtiapp.fi` and `https://backend.2.rahtiapp.fi` are different origins, because an origin is scheme + the full hostname + port. Sharing the `.2.rahtiapp.fi` suffix does not help.

The best solution is to avoid CORS altogether: don't give the backend a Route, and call it from the frontend's server using the internal name (`http://backend-api:8000`). Then the browser only ever sees one origin, no CORS is needed, and the backend is not on the internet.

If the backend does need a public Route:

```
app.use(
  cors({
    origin: ["https://frontend.2.rahtiapp.fi"],
    credentials: true,
  })
);
```

Three things that break CORS on Rahti

1. **`origin: "*"` and `credentials: true` do not work together.** This is not a matter of style: the browser rejects a response carrying `Access-Control-Allow-Origin: *` if the request was made in `credentials: "include"` mode. List the origins explicitly.
2. **The preflight request is forgotten.** A call carrying an `Authorization` header or `Content-Type: application/json` makes the browser send an `OPTIONS` request first. If the backend does not answer it with a 2xx and the right headers, the actual request is never sent. Test it separately:

```
curl -i -X OPTIONS https://backend.2.rahtiapp.fi/api/data      -H "Origin:
https://frontend.2.rahtiapp.fi"      -H "Access-Control-Request-Method: POST"      -H
"Access-Control-Request-Headers: content-type"
```

The expected response is 204 or 200 with `Access-Control-Allow-Origin` present.

3. **Authentication kills the preflight.** The browser does **not** send cookies or an `Authorization` header on a preflight request. If the app requires a login on every path (Basic Auth, middleware), `OPTIONS` gets a 401 and the whole call fails. Let `OPTIONS` through the authentication. Same trap as the health checks below.

> The browser console error "*No 'Access-Control-Allow-Origin' header is present*" usually means one of these three, not that the `cors()` call is missing entirely.

Health checks and Basic Auth

Rahti expects the pod to signal that it's ready. Two approaches:

```
# Open /health path — the best option
readinessProbe:
  httpGet: { path: /health, port: 8000 }

# The app is entirely behind authentication — an HTTP check would get a 401
readinessProbe:
  tcpSocket: { port: 3000 }
```

> **This is a real pitfall.** If the app (e.g. Next.js middleware or `proxy.ts`) requires Basic Auth on **every** path, the `httpGet` check gets a 401, the pod never reaches Ready, and the rollout hangs forever. Use either a `tcpSocket` check or exempt `/health` from authentication.

Ports

- The app must listen on `0.0.0.0`, not `127.0.0.1` — otherwise traffic can't reach it from outside the pod.
- The port number itself doesn't matter (3000, 8000, 8080), as long as the same value is used in the Dockerfile's `EXPOSE`, the Deployment's `containerPort`, the Service's `targetPort`, and the Route's `targetPort`.
- Don't use a port below 1024. A random UID cannot bind a privileged port without the `NET_BIND_SERVICE` capability, the only one Rahti lets you add back. Listening on 8080 is the easier route.

- Read the port from an environment variable where possible: `const PORT = process.env.PORT || 8080`.

7. PostgreSQL and pgvector on Rahti

> A database in the same project as your app: installation, persistent storage, pgvector, and local management with pgAdmin.

Contents

Before you install

A self-managed database on Rahti uses **community images that CSC does not support**. It's fine for demos, teaching, and light production use, but for critical data you should consider a managed database service.

Two things you must get right:

1. **Persistent storage (PVC) before you write any real data.** Without it, the entire database disappears every time the pod restarts.
2. **No Route for the database.** Postgres belongs on the internal network. A Route would expose it to the internet.

Installing from the console

1. **+Add → Container images**
2. **Image name from external registry:**

   > `quay.io/rh-aiservices-bu/postgresql-15-pgvector-c9s`

   This is a community build from Red Hat's AI Services BU with pgvector already included, and it works with OpenShift's random UID. Check for a newer tag before you pin a version.
3. **Name:** e.g. `postgresql-pgvector`

4. **Environment variables** (on the *Environment* tab after creating the Deployment, or directly in the form):

Variable	Value
`POSTGRESQL_USER`	`postgres`
`POSTGRESQL_PASSWORD`	a strong password
`POSTGRESQL_DATABASE`	`vectordb`

5. **Uncheck "Create a route".**

The password belongs in a Secret, not directly in the form:

```
oc create secret generic postgres-secret \
  --from-literal=POSTGRESQL_USER=postgres \
  --from-literal=POSTGRESQL_PASSWORD='<strong-password>' \
  --from-literal=POSTGRESQL_DATABASE=vectordb -n <project>

oc set env deployment/postgresql-pgvector --from=secret/postgres-secret -n <project>
```

Persistent storage (PVC)

In the browser: open Deployment → *Actions* → *Add Storage*

- Name: `pgvector-data`
- Access mode: **Single user (RWO)**
- Size: 5 GiB or more (the quota allows 100 GiB total by default)
- Mount path: `/var/lib/pgsql/data`

From the command line:

```
oc set volume deployment/postgresql-pgvector \
  --add --name=pgvector-data \
  --type=pvc --claim-name=pgvector-data \
  --claim-size=5Gi --claim-mode=ReadWriteOnce \
  --mount-path=/var/lib/pgsql/data -n <project>
```

Check it:

```
oc get pvc -n <project>
```

Installing with manifests

For a repeatable install, the same thing declaratively:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pgvector-data
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 5Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgresql-pgvector
spec:
  replicas: 1
  strategy:
    type: Recreate          # an RWO volume can't be shared by two pods
  selector:
    matchLabels: { app: postgresql-pgvector }
  template:
    metadata:
      labels: { app: postgresql-pgvector }
    spec:
      securityContext: {}   # NO runAsUser
      containers:
        - name: postgresql
          image: quay.io/rh-aiservices-bu/postgresql-15-pgvector-c9s
          envFrom:
            - secretRef: { name: postgres-secret }
          ports:
            - containerPort: 5432
          volumeMounts:
            - name: data
              mountPath: /var/lib/pgsql/data
          resources:
            requests: { cpu: 100m, memory: 512Mi }
            limits:   { cpu: 500m, memory: 1Gi }
      volumes:
        - name: data
          persistentVolumeClaim:
            claimName: pgvector-data
```

```
---
apiVersion: v1
kind: Service
metadata:
  name: postgresql-pgvector
spec:
  selector: { app: postgresql-pgvector }
  ports:
    - port: 5432
      targetPort: 5432
```

> `strategy: Recreate` matters: by default a rolling update would try to start a new pod before shutting down the old one, but an RWO volume can't be attached to two pods at once — the new pod would get stuck in `Pending`.

Enabling the pgvector extension

The extension is enabled once, inside the database:

```
CREATE EXTENSION vector;
```

Directly from the pod:

```
oc exec -it deployment/postgresql-pgvector -n <project> -- \
  psql -U postgres -d vectordb -c "CREATE EXTENSION IF NOT EXISTS vector;"
```

Or in pgAdmin: *Query Tool* (Ctrl+Shift+Q) → the same command → F5. Alternatively, right-click *Extensions* → *Create* → *Extension...* and pick `vector`.

Usage example:

```
CREATE TABLE items (id bigserial PRIMARY KEY, embedding vector(3));
INSERT INTO items (embedding) VALUES ('[1,2,3]'), ('[4,5,6]');

-- Nearest neighbors (L2 distance)
SELECT * FROM items ORDER BY embedding <-> '[3,1,2]' LIMIT 5;
```

For larger datasets you'll need an index:

```
CREATE INDEX ON items USING hnsw (embedding vector_cosine_ops);
```

Connecting from the app

Within the same project, the database is reachable by its service name:

```
DATABASE_URL=postgresql://postgres:<password>@postgresql-pgvector:5432/vectordb
```

```
oc create secret generic app-db \
  --from-literal=DATABASE_URL='postgresql://postgres:<password>@postgresql-
pgvector:5432/vectordb' \
  -n <project>
oc set env deployment/app --from=secret/app-db -n <project>
```

Local management via port-forward

Since there's no Route, the database is managed through a tunnel:

```
oc project <project>
oc port-forward svc/postgresql-pgvector 5432:5432
```

Leave the window open. Connect with pgAdmin or `psql`:

Field	Value
Host	`localhost`
Port	`5432`
Database	`vectordb`
Username	`postgres`
Password	`POSTGRESQL_PASSWORD`

```
psql postgresql://postgres:<password>@localhost:5432/vectordb
```

The port-forward is a temporary management connection — not how the app talks to the database.

Backups

Backups are **your responsibility**. Minimum viable approach:

```
# Dump to a local file
oc exec deployment/postgresql-pgvector -n <project> -- \
  pg_dump -U postgres vectordb > backup-$(date +%F).sql

# Restore
cat backup-2026-08-31.sql | oc exec -i deployment/postgresql-pgvector -n <project> -- \
  psql -U postgres -d vectordb
```

It's worth shipping dumps to Allas: 8. Allas S3 storage.

8. Allas object storage (S3)

> Rahti pods are stateless: data written to a pod's disk is lost. Persistent data belongs either on a PVC or — for large files, shared access, backups — in CSC's Allas object storage.

Contents

S3 vs Swift

Allas supports two protocols. **Use S3 in applications.**

	S3	Swift
Credential lifetime	long-lived keys	token expires after ~8 h
Fits Rahti	yes	poorly — the token would expire mid-run
Library support	boto3, AWS SDK, s3cmd, rclone	swift-client

Don't mix protocols within the same bucket.

Service	Endpoint	Region
S3 API	`https://a3s.fi`	`regionOne`
Swift API	`https://a3s.fi:443/swift/v1/AUTH_<project-id>`	`regionOne`

> The region is always `regionOne`, even though the server is in Finland. Not `eu-north-1`.

1. Create a bucket

1. Log in at allas.csc.fi
2. Select the right project on the left
3. **+ Create bucket**

Naming: bucket names must be unique **across all Allas users**. CSC recommends prefixing with your project number, e.g. `1234567-raw-data`. Lowercase letters, digits, and hyphens only — no special characters.

> **Three different "projects" — don't mix them up.** OpenStack's `project_XXXXXXX` is the same **CSC computing project** whose number you gave the Rahti project in the `csc_project` field (chapter 1). The Rahti project — the namespace — is a different thing: it's a namespace that lives inside the computing project.
>
> **Credentials and buckets are computing-project-scoped.** Make sure you create the credentials in the same project as the bucket — otherwise you'll get a 403 and won't know why.

2. Install the OpenStack tools

Credentials are obtained with the OpenStack command-line tool.

```
python -m pip install --user python-openstackclient
```

If the `openstack` command isn't found after installation, the Scripts folder isn't on your PATH. Find the correct path:

```
python -c "import sys, os; print(os.path.dirname(sys.executable) + '\\Scripts')"
```

Add the output to your PATH (permanently: Windows + R → `sysdm.cpl` → *Advanced* → *Environment Variables* → *Path* → *New*), open a new terminal, and verify:

```
openstack --version
```

Configuration:

1. Log in at pouta.csc.fi
2. **API Access** → **Download OpenStack RC File** → **OpenStack clouds.yaml file**
3. Save the file to `~/.config/openstack/clouds.yaml` (on Windows: `C:\Users\<username>\.config\openstack\clouds.yaml`)

```
mkdir "$HOME\.config\openstack"
```

The file content looks like this:

```
clouds:
  openstack:
    auth:
      auth_url: https://pouta.csc.fi:5001/v3
      username: "<csc-username>"
      project_id: "<project-id>"
      project_name: "project_XXXXXXX"
      user_domain_name: "Default"
      # password: leave this out — it's prompted for at run time
    regions:
      - regionOne
    interface: "public"
    identity_api_version: 3
```

3. Get S3 credentials

```
$env:OS_CLOUD = "openstack"

# Make sure you're in the right project
openstack configuration show          # check auth.project_id
openstack project list                # all projects
openstack project set project_XXXXXXX # switch if needed

# List existing credentials
openstack ec2 credentials list

# Create new ones if needed
openstack ec2 credentials create
```

The command prompts for your CSC password. Output:

```
+----------------------------------+----------------------------------+------------------
---------------+---------+
| Access                           | Secret                           | Project ID
| User ID |
+----------------------------------+----------------------------------+------------------
---------------+---------+
| abc123def456ghi789jkl012mno345pq | xyz789uvw456rst123qpo890lmn567abc |
def456abc789ghi012jkl345mno678pq | tunnus  |
+----------------------------------+----------------------------------+------------------
---------------+---------+
```

`Access` = access key, `Secret` = secret key.

> On CSC's supercomputers (Roihu, LUMI) the same thing is one command: `allas-conf` prints the S3 connection details directly.

4. Store the credentials

Locally, in a `.env` file (which is in `.gitignore`):

```
ALLAS_ACCESS_KEY_ID=abc123def456ghi789jkl012mno345pq
ALLAS_SECRET_ACCESS_KEY=xyz789uvw456rst123qpo890lmn567abc
ALLAS_ENDPOINT_URL=https://a3s.fi
ALLAS_BUCKET_NAME=1234567-raw-data
```

5. Usage from code

The single most important detail: Allas only supports **path-style** addressing (`a3s.fi/<bucket>/<key>`), not virtual-host style (`<bucket>.a3s.fi`). AWS SDK v3 defaults to virtual-host style, so you must override that setting.

Python (boto3)

```
import boto3, os

s3 = boto3.client(
    "s3",
    endpoint_url=os.getenv("ALLAS_ENDPOINT_URL"),
    aws_access_key_id=os.getenv("ALLAS_ACCESS_KEY_ID"),
    aws_secret_access_key=os.getenv("ALLAS_SECRET_ACCESS_KEY"),
    region_name="regionOne",
)

for b in s3.list_buckets()["Buckets"]:
    print(b["Name"])

bucket = os.getenv("ALLAS_BUCKET_NAME")
s3.upload_file("local.txt", bucket, "folder/remote.txt")
s3.download_file(bucket, "folder/remote.txt", "downloaded.txt")
```

boto3 handles path-style automatically for this endpoint. If you run into problems, force it:

```
from botocore.config import Config
s3 = boto3.client("s3", ..., config=Config(s3={"addressing_style": "path"}))
```

JavaScript / TypeScript (AWS SDK v3)

```
import { S3Client, ListBucketsCommand, PutObjectCommand } from "@aws-sdk/client-s3";

const s3 = new S3Client({
  endpoint: process.env.ALLAS_ENDPOINT_URL,   // https://a3s.fi
  region: "regionOne",
  forcePathStyle: true,                        // ← REQUIRED for Allas
  credentials: {
    accessKeyId: process.env.ALLAS_ACCESS_KEY_ID,
    secretAccessKey: process.env.ALLAS_SECRET_ACCESS_KEY,
  },
});

const { Buckets } = await s3.send(new ListBucketsCommand({}));

await s3.send(
  new PutObjectCommand({
    Bucket: process.env.ALLAS_BUCKET_NAME,
    Key: "test.txt",
    Body: "Hello Allas!",
  })
);
```

Credentials for Rahti

```
oc create secret generic allas-credentials \
  --from-literal=ALLAS_ACCESS_KEY_ID='...' \
  --from-literal=ALLAS_SECRET_ACCESS_KEY='...' \
  --from-literal=ALLAS_ENDPOINT_URL='https://a3s.fi' \
  --from-literal=ALLAS_BUCKET_NAME='1234567-raw-data' \
  -n <project>

oc set env deployment/app --from=secret/allas-credentials -n <project>
```

These are runtime variables — **don't put them behind a `NEXT_PUBLIC_` or `VITE_` prefix**, or the keys will end up in the browser (see 05 Environment variables).

If the build fails on missing keys, give it an empty placeholder:

```
ARG ALLAS_ACCESS_KEY_ID=""
ENV ALLAS_ACCESS_KEY_ID=$ALLAS_ACCESS_KEY_ID
RUN npm run build
```

Troubleshooting

Symptom	Likely cause
`SignatureDoesNotMatch`	Wrong secret key, or virtual-host addressing is on → set `forcePathStyle: true`
`403 Forbidden` on the bucket	Credentials were created in a different project than the bucket
`NoSuchBucket`	The bucket is in a different project, or the name has a typo
`openstack: command not found`	The Python Scripts folder isn't on PATH (see above)
`pip: command not found`	Use `python -m pip install …`
Connection times out from Rahti	Check the endpoint doesn't use `http://` — only `https://a3s.fi`

9. Troubleshooting

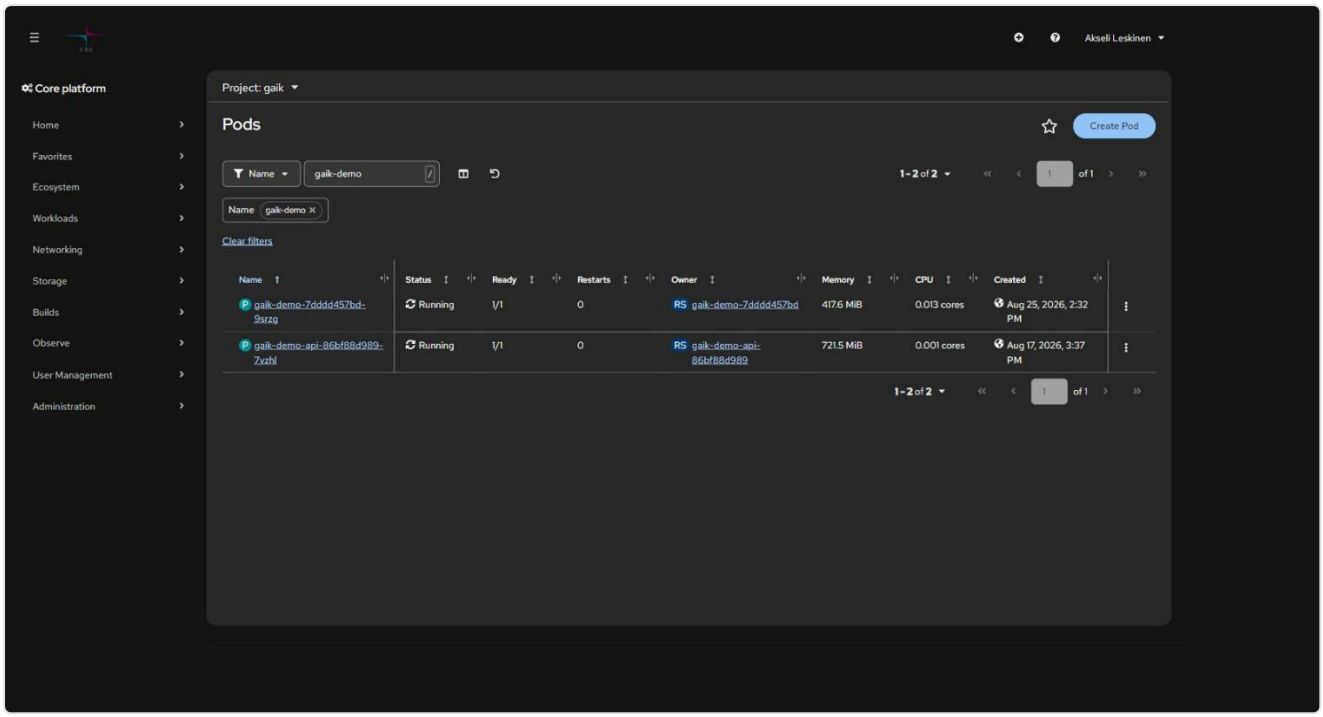
> From symptom to cause. Always start with the same three commands — don't guess.

Contents

The first three commands

```
oc get pods -n <project>                                    # 1. what state is it in
oc logs deployment/<app> -n <project> --tail=50              # 2. what does the app say
oc get events -n <project> --sort-by=.lastTimestamp | tail   # 3. what does the cluster say
```

The pod's state tells you which direction to go: `Running` but misbehaving → check logs. `Pending` / `CrashLoop` / `ImagePull` → check events and `oc describe`.



The same information is in the console: *Workloads → Pods → <pod> → Logs / Events*.

Pod states and what they mean

State	Meaning	Next command
`Running` + `1/1`	Running and ready	—
`Running` + `0/1`	Running, but the readiness check isn't passing	`oc describe pod <pod>`
`Pending`	Not scheduled — quota, PVC, or resource request	`oc describe pod <pod>`
`CrashLoopBackOff`	Starting and crashing repeatedly	`oc logs <pod> --previous`
`ImagePullBackOff`	The image can't be pulled	`oc describe pod <pod>`
`Error` / `Completed`	The process finished (job, build)	Normal for jobs
`Terminating` for a long time	Shutdown is stuck	`oc delete pod <pod> --force`

The Deployment doesn't update after a push

The most common source of confusion: the image has been pushed, but the old code is still running.

```
# 1. Did the new image arrive?
oc get is <imagestream> -n <project>
oc describe is <imagestream> -n <project> | head -30

# 2. Does an image trigger exist and is it enabled?
oc get deployment <app> -n <project> \
  -o jsonpath="{.metadata.annotations.image\.openshift\.io/triggers}"
```

If the output is empty or contains `"paused":"true"`:

```
oc annotate deployment/<app> image.openshift.io/triggers- -n <project>
oc set triggers deployment/<app> \
  --from-image=<imagestream>:latest -c <container> -n <project>
```

A quick fix without a trigger:

```
oc rollout restart deployment/<app> -n <project>
oc rollout status  deployment/<app> -n <project>
```

Or set the image manually:

```
LATEST=$(oc get is <imagestream> -n <project> \
  -o jsonpath='{.status.tags[0].items[0].dockerImageReference}')
oc set image deployment/<app> <container>=${LATEST} -n <project>
```

Also check the builds if you're using a BuildConfig — a failed build means a new image was never produced:

```
oc get builds -n <project>
```

CrashLoopBackOff

```
oc logs deployment/<app> -n <project> --previous   # log from the previous attempt
oc describe pod <pod> -n <project>                  # exit code and events
```

The usual causes:

Cause	Telltale sign
Missing environment variable	Log shows `undefined`, `KeyError`, `connection string is empty`
App listens on `127.0.0.1`	Starts normally, but the health check doesn't respond
Wrong port	`containerPort` ≠ the app's port
Needs root privileges	`permission denied`, `EACCES`, `mkdir failed`
Wrong architecture	`exec format error` → the image is arm64, amd64 is needed
OOM	Exit code 137

ImagePullBackOff / ErrImagePull

```
oc describe pod <pod> -n <project> | grep -A5 Events
```

- **Private registry without a pull secret** → create a secret and link it:

  ```
  oc secrets link default <pull-secret> --for=pull -n <project>
  ```

- **Wrong tag** → check that the tag exists: `oc get is <name> -o yaml`
- **Wrong address** → from inside the cluster: `image-registry.openshift-image-registry.svc:5000/<project>/<image>`, from outside: `image-registry.apps.2.rahti.csc.fi/<project>/<image>`

Pod stuck in Pending

```
oc describe pod <pod> -n <project> | tail -20
oc describe AppliedClusterResourceQuotas
```

Three typical causes:

1. **Quota exhausted.** The quota is shared across the whole computing project. Shut down unneeded apps or scale to zero: `oc scale deployment/<x> --replicas=0`
2. **The PVC can't get storage.** `oc get pvc -n <project>` — if the state is `Pending`, the size may exceed the quota (max 100 GiB / PVC).
3. **The resource request exceeds a limit.** A LimitRange defines the maximums; a `requests` value that's too large gets rejected.

OOMKilled (exit 137)

The container exceeded its memory limit.

```
oc adm top pods -n <project>
oc set resources deployment/<app> -n <project> --limits=memory=2Gi --requests=memory=512Mi
```

Remember the ratio limit: `limits` may be at most 5× `requests` (check your project's LimitRange). So just raising the ceiling without also raising the reservation can get rejected.

Permission denied when writing a file

Rahti gives the container a random UID, but the group is always **0**. Make writable directories writable by group 0:

```
RUN mkdir -p /app/output && \
    chown -R 1001:0 /app/output && \
    chmod -R g+rwX /app/output
USER 1001
```

The same applies to `.next`, `tmp`, and cache directories. If the app writes to the root directory, redirect it to write to `/tmp` instead — that's always writable.

Do not try to fix this with a `runAsUser` setting in the manifest; the SCC rejects it.

500 error on docker push

```
unknown: unexpected status from HEAD request to
https://image-registry.apps.2.rahti.csc.fi/v2/<project>/<image>/manifests/sha256:… : 500
```

The ImageStream is missing. Create it first:

```
oc create imagestream <image> -n <project>
```

Also check that you're logged in to the registry (with the `oc whoami -t` token) and that the image is under 5 GiB.

504 Gateway Time-out

HAProxy's default timeout is 30 seconds. See 04 Routes and networking.

```
oc annotate route <route> haproxy.router.openshift.io/timeout=120s -n <project> --overwrite
```

The app doesn't respond through the Route

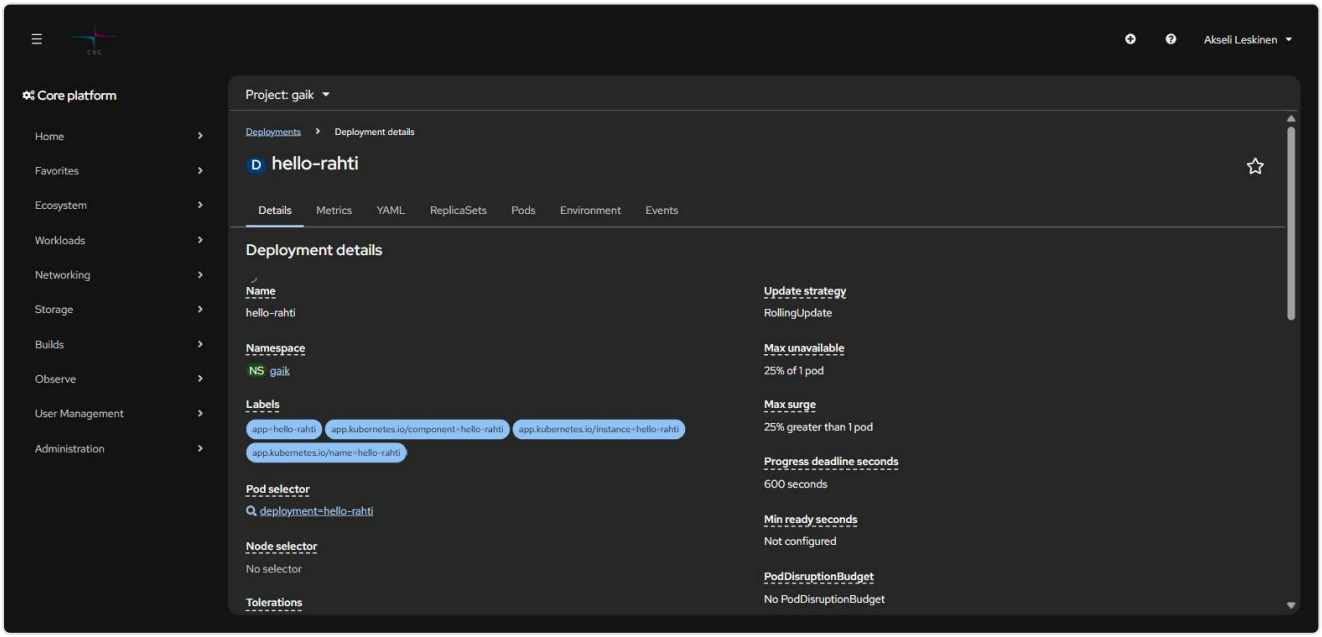
Work from the outside in:

```
# 1. Does the Route exist and is it admitted?
oc get route <route> -n <project>

# 2. Does it point to the right service and port?
oc describe route <route> -n <project>

# 3. Does the Service respond?
oc get svc <service> -n <project> -o wide
oc get endpoints <service> -n <project>      # empty = the label selector doesn't match any pods

# 4. Does the pod respond directly?
oc port-forward deployment/<app> 8080:<port> -n <project>
curl -I http://localhost:8080
```

An empty `endpoints` list is very common: the Service's `selector` doesn't match the pod's labels.

A classic case: `oc new-app` labels pods `deployment=<name>`, not `app=<name>`. That's why `oc get pods -l app=<name>` returns "No resources found" even though the pod is up, and a hand-written `selector: {app: <name>}` leaves the Service empty. Check the actual labels before you guess:

```
oc get pods -n <project> --show-labels
oc get svc <service> -n <project> -o jsonpath='{.spec.selector}'
```



The console shows the same on the Deployment's *Details* tab under **Pod selector**.

Unauthorized / login expired

```
error: You must be logged in to the server (Unauthorized)
```

A personal token expires after about a day. First check who you are:

```
oc whoami
oc config current-context
```

If the context has switched back to your personal account even though you should be using a service account, log in with it again. The permanent fix is a service account with a long-lived token: 1. Getting started.

False alarms

These look like problems but aren't:

- **A restart count by itself is not an alarm.** A pod that's been running for months accumulates restarts from normal recycling. Weigh it against the pod's age: 40 restarts over 90 days is not the same as 40 restarts in an hour.
- **HTTP 401/403 on a Route means the app is responding.** It's up and requires authentication. Only 5xx, connection errors, and timeouts are alarms.
- **The first request to a rarely used app can time out** (cold start). Try again before marking it as down.
- **Pods in `Completed` state are not failures** — they're finished job, cronjob, and build runs.
- **An empty `resourcequota`** is normal: the quota lives at the computing-project level. Use `oc describe AppliedClusterResourceQuotas`.
- **A live Deployment shows `@sha256:…` instead of `:latest`** — the image trigger has resolved the tag to a digest. That's how it's supposed to work.

Whole-namespace status overview

When you want to know in one shot whether everything is up:

```
oc get all -n <project>
oc get events -n <project> --field-selector type=Warning --sort-by=.lastTimestamp
```

This repository also ships an **`rahti-audit` skill**, which gathers the same status overview in a single run (deployments, pods, routes with their HTTP responses, warning events, and quotas) and reports

only the anomalies. See 10. Agentic development.

Previous: 8. Allas S3 · **Next:** 10. Agentic development →

10. Agentic development: skills for working with Rahti

> This repository includes four **agent skills** that teach an AI agent (Claude Code, Codex, Cursor, etc.) to use CSC's environments correctly. A skill is just a markdown file — nothing to install, no API key.

Contents

What a skill is

A skill is a folder containing `SKILL.md` and, optionally, additional files:

```
csc-rahti/
├── SKILL.md              # instructions for the agent + a description of when to use it
├── references/           # deeper reference docs, read only when needed
│   ├── routes.md
│   ├── authentication.md
│   └── …
├── scripts/               # runnable helper scripts
└── tests/                 # tests for the scripts
```

At the top of `SKILL.md` is a YAML block whose `description` field tells the agent **when** it's worth loading the skill. The agent reads the descriptions continuously, but only reads the full content once it's relevant — that way even dozens of skills don't fill up the context window.

The practical benefit: without the skill, the agent guesses Rahti's details wrong (outdated console navigation, `runAsUser` in a manifest, a forgotten ImageStream). With the skill, it knows the house conventions.

The repository's skills

Skill	For	When it triggers
csc-rahti	Rahti deployment, images, routes, secrets, Satama, Allas, the Aitta LLM API	"deploy to Rahti", "oc", "rahtiapp.fi", "imagestream", "satama"
rahti-audit	Read-only status overview of a whole namespace	"is production okay", "audit Rahti", "crashloop"
csc-roihu	The Roihu supercomputer: Slurm, GH200 GPUs, modules	"sbatch", "run on GPU", "supercomputer"
csc-lumi	LUMI: AMD MI250X, ROCm, the LUMI software stack	"LUMI", "ROCm", "MI250X"

The division of labor is deliberate: `rahti-audit` is **read-only** and must never fix anything; `csc-rahti` makes changes. That way "check what's broken" can never accidentally alter production.

A quick guide to choosing between CSC services:

Need	Service	Skill
Stand up a web app or API	Rahti	csc-rahti
Train a model or run a large batch job on NVIDIA GPUs	Roihu	csc-roihu
The same on AMD GPUs, with a EuroHPC quota	LUMI	csc-lumi
A ready-made LLM endpoint without your own GPU	Aitta	csc-rahti (Aitta section)

Installation

Option 1: clone the repository (easiest)

When you open this repository with Claude Code, the skills are available right away — they live in `.claude/skills/`, which Claude Code reads on a per-project basis.

```
git clone <this-repo-url>
cd csc-rahti-guide
claude
```

Option 2: copy them for personal use (all projects)

Copy the skills you want into your personal skills folder, and they'll be available in every project:

```
# macOS / Linux
cp -r .claude/skills/csc-rahti ~/.claude/skills/
cp -r .claude/skills/rahti-audit ~/.claude/skills/
```

```
# Windows PowerShell
Copy-Item -Recurse .claude\skills\csc-rahti  "$HOME\.claude\skills\"
Copy-Item -Recurse .claude\skills\rahti-audit "$HOME\.claude\skills\"
```

Check the result in Claude Code with the `/skills` command.

Option 3: shared `.agents/skills` folder

Some agent tools read skills from `.agents/skills/` (the agentskills.io convention). The same `SKILL.md` works for both, so there's no need to duplicate files — make a link instead:

```
# macOS / Linux
mkdir -p ~/.agents/skills
ln -s ~/.claude/skills/csc-rahti ~/.agents/skills/csc-rahti
```

```
# Windows: a directory junction (does not require administrator rights)
New-Item -ItemType Junction -Path "$HOME\.agents\skills\csc-rahti" `
         -Target "$HOME\.claude\skills\csc-rahti"
```

> **Claude Code does not read `.agents/skills`** — for it, `~/.claude/skills/` or the project's `.claude/skills/` is enough. The link is there for other tools, and points that way round so that the Claude Code path stays the original.
>
> If your tool reads its instructions from an `AGENTS.md` file, add a line there pointing to this repository's `docs/` folder and skills.

Dependencies

The skills themselves require nothing, but the commands they recommend do:

Tool	For	Check
`oc`	all Rahti operations	`oc version --client`
`docker` or `podman`	building images	`docker --version`
PowerShell 7	the `csc-rahti` and `rahti-audit` scripts	`pwsh --version`
`openstack`	Allas credentials	`openstack --version`

Usage

Invoke it by name (Claude Code, slash command):

```
/csc-rahti How do I get an address like demo.2.rahtiapp.fi for my app?
```

Or just describe the task — the skill triggers on its own based on its description:

```
Deploy this Next.js app to Rahti in the project my-project,
port 3000, and set up a route for it.
```

```
Check whether anything is broken in Rahti.
```

```
gaik-demo returns a 504 when a query takes more than half a minute. Fix it.
```

A typical agent workflow for Rahti:

1. **Log in yourself** — the agent can't get through MFA (see below). Run `oc login` or `Connect-Rahti.ps1`.
2. **Give it the task.** The agent reads the skill, checks the current state with `oc get` commands, and proposes changes.
3. **Review the proposal** before approving any write commands.
4. **Let it run the deploy** and read the `oc rollout status` output.

UI vs. CLI vs. agent

Task	Recommendation	Why
First-time exploration	**UI**	You see what objects get created
A one-off demo deployment	**UI**	Fastest, no files needed
Repeated deployment	**CLI + manifests**	Version-controlled, repeatable
Quick change to an environment variable	**UI** or `oc set env`	Both trigger a rollout
Creating secrets	**CLI**	`--from-literal` is precise; the form doesn't show the result
Reading logs	**UI** for light use, `oc logs -f` for serious use	The UI truncates long logs
Production status overview	**Agent** (`rahti-audit`)	Ten commands in one run, reports only the anomalies
Troubleshooting	**Agent + CLI**	The agent knows the right order to check things; you decide the fix
Raising quotas, permissions	**UI + Service Desk**	Requires a human

Rule of thumb: **UI for learning and one-off tasks, CLI for repeatable work, agent for speeding up routine tasks.** An agent doesn't replace understanding what it's doing — that's why it's worth reading chapters 1–9 even if you let an agent do the work.

What the agent can't do

- **Get through CSC's MFA.** A personal login is always a human task. The agent can use an already-created service account, but it can't create the first credential without you having logged in.
- **Fetch a token from the web console.** *Copy login command* requires a browser session.
- **Raise a quota or grant Rahti permissions** — those go through MyCSC and the Service Desk.
- **Know project-specific limits without checking.** Have it run `oc describe limitranges` instead of trusting default assumptions.

Safety rules

The skills are written according to these rules, and the same rules apply to you:

1. **Tokens are never printed.** Not to chat, not to logs, not to a commit. Scripts pass the token straight to `oc`.
2. **Secrets stay out of the repository.** Service account tokens are stored in a separate env directory, not in the skill and not in the project.
3. **Read-only stays read-only.** `rahti-audit` never runs `oc delete`, `oc scale`, or `oc apply` — even when the fix looks obvious.
4. **Write commands against production are confirmed.** `oc delete`, `oc scale --replicas=0`, and `oc apply` are things worth reading before approving.
5. **Least privilege.** `view` for auditing, `edit` for deployment, `admin` only if managing permissions is actually needed.

Editing the skills

The skills are deliberately readable markdown — adapt them to your own environment:

- Swap the `<namespace>` placeholders for your own project's name if you only work in one.
- Add your own organization's conventions to the `references/` folder.
- Keep the `description` field descriptive: it determines whether the skill triggers at the right time.
- The scripts are tested — if you change them, run the tests:

```
pwsh -NoProfile -Command "& '.claude/skills/csc-rahti/tests/RahtiCredentials.Tests.ps1'"
```

Do not commit project-specific namespaces, credentials, customer names, or tokens into a skill if the repository is public.

Skills across multiple machines

If you work across multiple machines, don't copy skills from one machine to another — they'll drift apart within a week. Keep one folder as the single source of truth and link to it:

```
# A folder that syncs between machines (OneDrive, Dropbox, a git repo…)
$src = "$HOME\OneDrive\agents-setup\skills"

# Claude Code reads this path
New-Item -ItemType Junction -Path "$HOME\.claude\skills" -Target $src
```

```
# macOS / Linux
ln -s ~/Sync/agents-setup/skills ~/.claude/skills
```

The same folder can serve several tools at once: one source, many links. Secrets don't belong in this folder — keep them separate (e.g. `agents-setup/env/`), so that sharing skills never shares tokens along with them.

A git repo as the sync folder is the best option for a team: changes are reviewable and the history is visible.

Further reading

Skills in general

- What are Skills? — Anthropic's overview
- Claude Code: Skills — the `SKILL.md` format, frontmatter fields, and lookup paths
- agentskills.io — the tool-agnostic specification this repository's skills follow

Packaging skills for wider distribution

If skills start to pile up and you want to share them across an organization, the next step is packaging them as a plugin — skills, tools, and configuration bundled as one installable unit:

- Agent plugins: package your skills, tools and more — what a plugin contains and when it's worth it
- Claude Code: Plugins — plugin structure and installation

> As of this writing, the `agent-plugins.org` specification page doesn't respond with a valid TLS certificate (the browser warns you), so the links above are the working sources.

Why isn't this repo packaged as a plugin? It was considered and rejected, for three reasons:

1. A Claude Code plugin reads skills from `skills/` at the plugin root, not from `.claude/skills/`. The skills would have to be either moved, which would stop a plain clone from loading them, or duplicated, which lets two copies drift apart.
2. The plugin format is Claude Code specific. It does nothing for a Copilot, Cursor or Codex user, and the whole point of this repo is that one file serves all of them.
3. `cp -r` is not a real obstacle for anyone.

If distribution ever grows, packaging is worth doing then. The calculation changes once a tool-independent specification is settled enough that one package serves every agent.

CSC's own documentation

- Rahti · Satama · Allas
- Roihu · LUMI · Aitta

Previous: 9. Troubleshooting · **Back:** Home